

IQuest-Q1: Advancing Agentic CLI Systems Towards Human-on-the-Loop RSI

IQuest Research

Abstract

We present IQuest-Q1, a sparse Mixture-of-Experts (MoE) agentic foundation model with 320B total parameters (15B activated). IQuest-Q1 is developed through a three-stage process: (1) *stable and efficient general pre-training*, which combines Stable SplitNorm, Muon, and μP with a two-stage curriculum whose second stage increases the coding-data mixture, preserving a wide stability margin between safe and optimal learning rates and achieving a 1.93 $\times$ gain in scaling efficiency over baseline; (2) *fine-grained mid-training*, expanding the data mixture across domains and formats and progressively extending the context; and (3) *multi-scaffold post-training*, combining supervised fine-tuning, domain-specialized RL, and multi-teacher on-policy distillation across various scaffolds to integrate complementary training signals and strengthen agentic coding capabilities. During this process, IQuest-Q1 participates in an iterative model-development loop with our infrastructure and contributes to its own improvement under human supervision. This human-on-the-loop process serves as an initial exploration of recursive self-improvement, with the model contributing to the development of subsequent model iterations under human supervision. The resulting IQuest-Q1 achieves advancing performance against comparable-scale models across coding and general agent benchmarks. We release IQuest-Q1 as an open-source model to support community research and development.

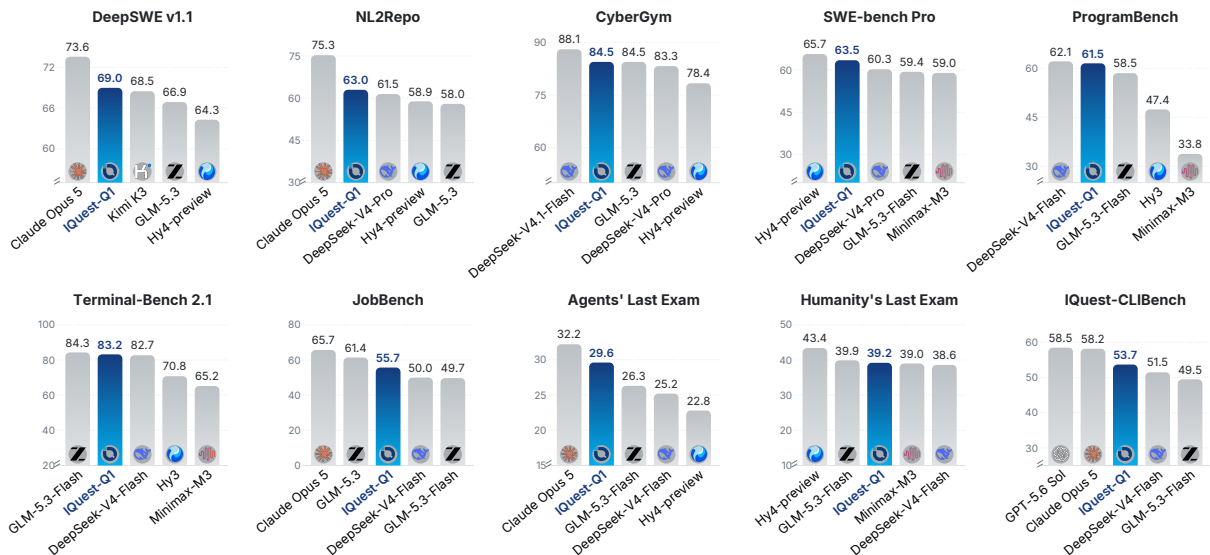


Figure 1. IQuest-Q1 performance across different benchmarks. DeepSeek-V4-Flash/Pro refer to the official 0731/0813 releases, respectively. Humanity's Last Exam is reported without tools. IQuest-CLIBench is our built-in benchmark for evaluating user experience.

1. Introduction

LLM-based agents [1–3] allow language models [4–10] to perform tasks through iterative reasoning, action, and interaction with external environments, rather than merely generating responses [1, 11, 12]. Among these environments, command-line interfaces (CLIs) have become one of the most general ways for language models to interact with external systems, use tools, access files, and execute code. As a result, building better agentic models calls for stronger CLI capabilities and, recursively, enabling models to participate in their own iterative improvement toward recursive self-improvement (RSI).

In this work, we release IQuest-Q1, an open-source agentic foundation model with broad CLI capabilities across a wide range of general and coding tasks, with particular strengths in software engineering, general-purpose task execution, long-horizon reasoning and execution, and cybersecurity. Across benchmarks covering these settings, IQuest-Q1 consistently ranks among the best-performing models of comparable scale. Beyond benchmark performance, we also optimize IQuest-Q1 for real-world CLI use, with reliable tool use, consistent behavior across different agent harnesses, and stable performance on long-horizon workflows.

Methods. IQuest-Q1 is trained through a multi-stage pipeline, as illustrated in Figure 2, to progressively develop general knowledge, long-context capabilities, and agentic CLI capabilities. General pre-training combines Stable SplitNorm [13, 14], Muon [15], and μP parameterization [16] with a two-stage curriculum that increases the coding-data mixture in the second stage. This design improves scaling efficiency while maintaining a wide stability margin between safe and optimal learning rates, achieving a $1.93\times$ gain in scaling efficiency over the baseline. Mid-training then expands the data mixture across domains and formats while progressively extending the context.

Post-training (shown in Figure 6) builds diverse atomic CLI [17, 18] capabilities through synthesized tasks executed in production environments. Specifically, SFT establishes basic CLI interaction, while RL enhance atomic capabilities through task-level feedbacks and multi-teacher on-policy distillation (short as MOPD) consolidates complementary model capabilities from frozen diverse experts into a unified model. Model merging further combines complementary capabilities from expert models trained for different CLIs, while mitigating conflicts among their parameter updates.

Results. Figure 1 shows the results of IQuest-Q1 on 10 agentic and reasoning benchmarks: (1) Strong coding capability. IQuest-Q1 demonstrates strong performance across coding benchmarks, including SWE-bench Pro, DeepSWE v1.1, CyberGym, and NL2Repo. (2) Broad general agentic and reasoning capability. Beyond coding, IQuest-Q1 performs well on general agentic and reasoning benchmarks, including JobBench, Agents’ Last Exam, and Humanity’s Last Exam. (3) Optimized CLI interaction experience. On our internal IQuest-CLIBench, IQuest-Q1 offers an optimized CLI interaction experience for interactive tasks.

Beyond benchmark performance, IQuest-Q1 also participates in an iterative model-development loop with our internal infrastructure developers, contributing to the development of its own successors under human supervision. This human-on-the-loop process forms a practical recursive self-improvement (RSI) loop, allowing model-generated improvements to accelerate subsequent model iterations.

Our contributions are summarized as follows:

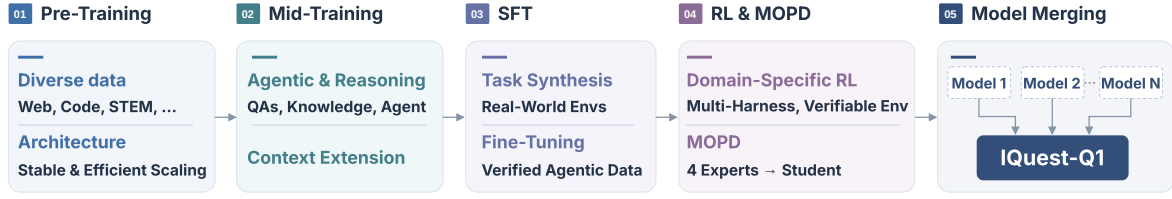


Figure 2. Training pipeline of IQuest-Q1.

- **IQuest-Q1: an open-source agentic model with frontier-level performance.** IQuest-Q1 is a sparse MoE model with 320B total parameters, of which 15B are activated per token. It combines strong general-purpose, coding-agent, and CLI interaction capabilities, achieving competitive performance against substantially larger and proprietary models on coding and general agent benchmarks.
- **CLI-centric training pipeline.** We build the training pipeline around CLI-based agentic interaction. Pre-training progressively develops the capabilities required for downstream interaction through staged data mixtures. Post-training specializes these capabilities through task and environment synthesis, multi-scaffold RL, and MOPD to consolidate multiple models into a unified model.
- **Model development with human-on-the-loop RSI.** We establish an RSI loop in which IQuest-Q1 evaluates its own capabilities, identifies potential improvements, and initiates training through a unified infrastructure interface under human supervision. This loop enables continuous model improvement while shortening the overall development cycle.

2. Model Architecture

2.1. Overall Architecture

IQuest-Q1 builds on the Transformer backbone [19], augmented with sparse Mixture-of-Experts (MoE) layers [20] and hybrid sliding-window attention. The attention stack interleaves Grouped-Query Attention (GQA) [21] and Sliding Window Attention (SWA) [22]. Within each attention module, a query-aware sink token [23] provides an adaptive destination for excess attention mass. Query-key normalization (QKNorm) stabilizes attention by controlling query and key scales [24], and QKNorm Clip is additionally applied to prevent excessively large affine parameters. Partial RoPE applies rotary position embeddings [25] to only a subset of the dimensions in each attention head. On the feed-forward side, the first Transformer layer replaces sparse MoE routing with a dense MLP to mitigate expert load imbalance in the earliest layer of the model.

Beyond the conventional MoE architecture, IQuest-Q1 introduces coordinated improvements to normalization and optimization. Stable SplitNorm is adopted to increase the model’s effective depth [26, 27], while the Muon optimizer [15] is combined with μP parameterization [16] to support stable scaling. Figure 3 provides a global overview of the resulting architecture.

Stable SplitNorm. Traditional Pre-Norm [28] may underutilize the depth of the model. As the unnormalized residual stream accumulates across layers, its variance can grow with depth, reducing the relative contribution of later residual updates [29]. Motivated by these observations, IQuest-Q1 adopts a hybrid normalization design introduced in prior work [13, 14]. We refer to this design as *SplitNorm*, which splits the normalization strategy across Transformer sublayers: the residual stream entering the attention sublayer is normalized, whereas the residual stream entering the feed-forward network (FFN) is kept unnormalized. This design combines the

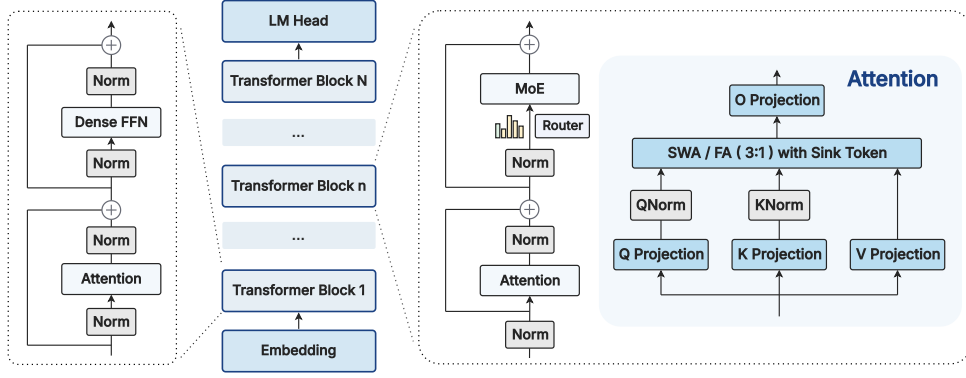


Figure 3. Overall architecture of IQuest-Q1.

stability benefits of Pre-Norm with the stronger depth utilization associated with Post-Norm, mitigating the underutilization of deeper layers in Pre-Norm architectures. Given an input x , the SplitNorm block used here is formulated using RMSNorm [30] as

$$z = \text{RMSNorm}(x), \quad h = z + \text{Attention}(z), \quad y = h + \text{FFN}(\text{RMSNorm}(h)). \quad (1)$$

This hybrid design combines normalized attention states with an identity residual path around the FFN. However, directly scaling this formulation to large models can lead to unstable optimization. To address this issue, IQuest-Q1 further introduces two modifications. First, an RMSNorm is applied to the attention output to prevent activation explosion, such that the attention update in Equation 1 becomes $h = z + \text{RMSNorm}(\text{Attention}(z))$. Second, a SandwichNorm structure [31], implemented with RMSNorm, is used for the first layer to stabilize the model input at the beginning of the residual stream. The resulting architecture is referred to as *Stable SplitNorm (SSN)*.

μP : Transfer Across Width and Depth. IQuest-Q1 adopts μP parameterization [16] for width-wise learning-rate transfer. Depth-wise transfer depends on additional parameterization choices, as studied in Depth- μP [32] and CompleteP [33]. Under our adopted parameterization, empirical validation demonstrates hyperparameter transfer across models spanning a $4\times$ range in width and a $4\times$ range in depth. Rather than tuning the learning rate directly at the target scale, the search is performed on a small proxy model with a hidden width of 896 and 18 Transformer layers. The learning rate selected on this proxy is then transferred zero-shot to the target model under the adopted parameterization, without additional learning-rate tuning at the target scale. We provide detailed μP scaling factors in Table 3 in the appendix.

Recursive Multi-token Prediction. To improve backbone learning through additional future-token supervision, we incorporate two multi-token prediction (MTP) [34, 35] layers from the start of pre-training. After completing the training pipeline, we adapt the first layer into a recursive drafter for speculative decoding and discard the second layer in a dedicated training stage. Similar to the draft model in EAGLE-3 [36], we extend the speculation horizon with a fixed parameter budget by reusing the same MTP layer across seven drafting steps [7, 37]. In each drafting step, the output hidden state and generated token are fed back into itself to produce subsequent predictions. Besides, we replace its attention with SWA using a smaller window, which bounds the per-step attention cost as the context grows. Details can be found in Figure 12 in the appendix.

2.2. Scaling Efficiency and Stability

Scaling efficiency. To evaluate scaling efficiency, we conduct a controlled comparison between IQuest-Q1 and a representative Qwen3-MoE-style design [38] following the Qwen3-30B-A3B configuration, using identical training data. As shown in Figure 4a, IQuest-Q1 consistently achieves lower training cross-entropy loss at a given training FLOPs budget. Overall, IQuest-Q1 exhibits a $1.93\times$ higher scaling efficiency than the baseline, requiring $1/1.93 \approx 52\%$ of the training compute to reach the same training loss. These results demonstrate the cumulative benefit of the proposed model design under controlled training conditions.

Scaling stability. Inspired by the Critical Batch Size (CBS) perspective [39, 40], we propose a complementary framework for characterizing the scaling limit imposed by training stability. CBS identifies the largest batch size that can be used without substantially degrading optimization efficiency; analogously, our framework aims to identify the largest model size that can be trained under a given architecture and optimization recipe without sacrificing either stability or optimization efficiency.

For a model of size N , we consider two learning-rate boundaries: the *optimal learning rate* $\eta_{\text{opt}}(N)$, which gives the most efficient optimization under a fixed training budget, and the *safe learning rate* $\eta_{\text{safe}}(N)$, defined as the largest learning rate that remains within the stable training regime. We provide the definition and evaluation criteria for stable training in Appendix A. The available stability headroom can be quantified as

$$H_N = \log \frac{\eta_{\text{safe}}(N)}{\eta_{\text{opt}}(N)}. \quad (2)$$

A positive H_N indicates that the efficiency-optimal operating point remains within the stable regime, while $H_N \leq 0$ indicates that optimization has become stability-constrained. We therefore define the *Critical Model Size (CMS)* N_{CMS} as the model scale at which the optimal and safe learning rates coincide:

$$\eta_{\text{opt}}(N_{\text{CMS}}) = \eta_{\text{safe}}(N_{\text{CMS}}). \quad (3)$$

This provides a compact measure of how far a given architecture and training recipe can scale while simultaneously maintaining training stability and optimization efficiency, serving as a proxy metric for the model’s scaling stability.

Figure 4b shows the scaling trends of both learning rates observed in our experiments. For IQuest-Q1, η_{safe} remains substantially above η_{opt} across the measured scales. Extrapolating the fitted trends yields $N_{\text{CMS}}^{\text{act}} \approx 572\text{B}$ and $N_{\text{CMS}}^{\text{total}} \approx 12\text{T}$, where $N_{\text{CMS}}^{\text{act}}$ and $N_{\text{CMS}}^{\text{total}}$ denote the CMS measured by activated and total parameters, respectively. These results indicate substantial stability headroom for scaling IQuest-Q1 beyond the model sizes studied.

2.3. Infrastructure

Data Packing. Sequence packing reduces padding overhead in language-model training [41]. To improve effective token throughput at a fixed sequence length and global batch size, we perform best-fit packing to the globally shuffled data stream using a bounded look-ahead window. The window contains up to (2BS) tokens, where (B) denotes the global batch size and (S) the sequence length. Samples left unpacked in the current window are carried over and prioritized in the next iteration, preventing them from being indefinitely delayed by subsequent packing decisions. Long-context training amplifies compute imbalance arising from document-length variation. Packed sequences are therefore reordered across data-parallel replicas to balance computation at micro-batch granularity.

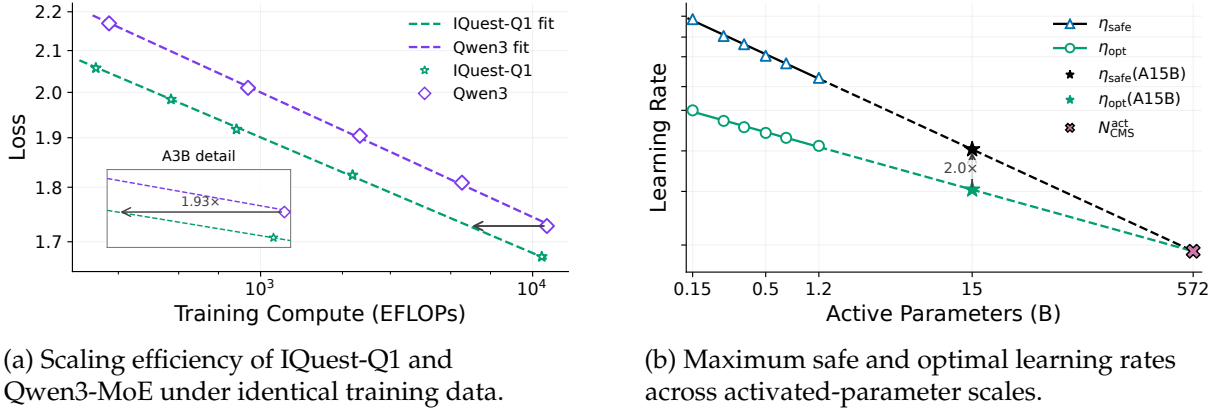


Figure 4. Scaling efficiency and stability of IQuest-Q1.

Sink Attention and Context Parallelism. Context parallelism uses ring-based K/V exchange for full attention [42], while sliding-window attention exchanges only the required halo and overlaps these exchanges with computation in both the forward and backward passes. During halo exchange, each rank computes attention over its local K/V, producing outputs for interior queries and partial outputs with associated Log-Sum-Exp (LSE) statistics for boundary queries. Once the halo arrives, the rank computes the complementary contributions for boundary queries and combines the local and halo partial outputs through LSE-based renormalization. This allows boundary queries to make progress before remote K/V becomes available while preserving sliding-window attention semantics. We illustrate the halo exchange schedule and partial output merging in Figure 11 in the appendix. Both full and sliding-window attention execution paths fuse learnable sink contributions into attention computation to reduce memory traffic.

Correctness Validation. To reduce silent errors introduced by model and training changes during RSI, the agent mines semantic invariants from the model’s training setup. These invariants are converted into checks that run during distributed training to detect deviations from single-machine semantics and violations of predefined correctness constraints.

3. Pre-Training

3.1. Pre-Training Data

We construct a high-quality, multilingual pre-training corpus spanning diverse domains, drawing from web pages, public GitHub code, books, academic papers, news, and domain-specific materials. For the code component, we preserve diversity across programming languages, modalities, and task granularities, dimensions emphasized by prior work on multilingual multimodal code generation, multi-granularity evaluation, and repository-level executable completion [43–45]. Our unified data processing pipeline includes cleaning, quality filtering, exact deduplication, and fuzzy deduplication to reduce noise and redundancy while preserving diversity.

General pre-training proceeds in two stages with a progressively adjusted data mixture. **Stage 1** establishes foundational language capabilities and general knowledge through diverse corpora, maximizing unique token counts across multiple domains. **Stage 2** gradually increases the proportion of code and STEM data to strengthen programming, mathematical, and scientific problem-solving capabilities. Specifically, we design a 4-phase pipeline increasing code data percentage from 40% to 70%.

Table 1. Summary of the main model architecture parameters.

Configuration	Value
Total Parameters	320B
Activated Parameters	15B
Transformer Layers	88
Hidden Dimension	3,072
Attention Heads (Q/KV)	48/8
Head Dimension	128
Hybrid Attention Pattern	3 SWA + 1 FA
Sliding Window Size	4,096
Partial RoPE Dimensions	32
Experts (Total/ Activated)	256/8
MTP Layers	2 Independent (Training) 1 Recursive $\times$ 8 (Inference)
MTP Sliding Window Size	512 (Inference)

Mid-training then combines high-quality natural corpora, QA pairs, and agentic trajectories to prepare the model for downstream reasoning and agentic tasks. Across pre-training and mid-training, the model is trained on a total of 29T tokens.

3.2. Hyper-Parameters

Model Hyper-Parameters. As summarized in Table 1, IQuest-Q1 has 88 Transformer layers and a hidden dimension of 3072. Attention uses 48 query heads and 8 key-value heads, each with a dimension of 128. The first and last two layers use full attention (FA) to capture global information and improve training stability, while the intermediate layers repeat three SWA layers followed by one GQA layer, yielding a 3 : 1 local-to-global ratio within each repeating block. The SWA uses a window size of 4,096. Partial RoPE is applied to 32 dimensions of each attention head, with RoPE bases of 10,000 and 1,000,000 for SWA and GQA, respectively. Query-Aware Sink Attention is enabled in all attention layers.

The first Transformer layer uses a dense FFN with an intermediate dimension of 12,288, while the remaining layers use sparse MoE FFNs. Each MoE layer contains 256 routed experts, with 8 experts activated per token and an intermediate dimension of 1,536 for each expert. Both dense and expert FFNs adopt SwiGLU. The vocabulary size is 160,000, with untied input embeddings and output weights.

During pre-training, IQuest-Q1 adopts two Multi-Token Prediction (MTP) layers to improve the performance of the backbone model. Each prediction order uses an embedding dimension of 128 and 8 attention heads, and the MTP loss is weighted by 0.3.

Training Hyper-parameters. We employ Muon [15] for all parameters except token embeddings, the language modeling head, normalization parameters, and MoE routers, which are optimized with Adam. For Adam, we set $\beta_1 = 0.9$, $\beta_2 = 0.95$, and $\epsilon = 10^{-15}$. For Muon, we set the momentum to 0.95. For an $m \times n$ weight matrix, we scale the orthogonalized update by $0.2\sqrt{\max(m, n)}$, combining spectral scaling with an additional factor of 0.2 to align the update RMS with that of Adam-style updates [15]. We apply weight decay with a coefficient of 0.1 to all parameters except layer normalization parameters, which are exempt from weight decay. We

additionally apply QKNorm Clip, capping the affine parameters of QKNorm at 10. We adopt a warmup-stable-decay learning rate schedule. The learning rate is warmed up to 6×10^{-4} and held constant for the remainder of Stage 1, then decays to 2×10^{-4} by the end of Stage 2 and to 6×10^{-5} by the end of mid-training.

3.3. Mid-Training and Long-Context Extension

3.3.1. Training Curriculum

Mid-training bridges broad pre-training and specialized agent post-training through a curriculum that combines a diverse data mixture, selective token-level supervision, and progressive context extension.

Data Organization. The mid-training mixture comprises three principal categories: high-quality natural corpora, QA pairs [46–48], and agentic trajectories [49, 50]. We organize training data using a chat template adapted for mid-training, retaining a selected subset of special tokens, including those associated with reasoning and tool calls.

Token-Level Loss Masking. We specify loss masks directly for individual tokens, independently of the role-specific special tokens used to determine supervision in a typical post-training setup. This decouples data formatting from loss computation and allows supervision to be tailored to selected spans within each training example, with finer control than masking entire conversational roles.

Progressive Context Extension. We extend the maximum training context lengths through 32K, 128K, and 512K tokens, using variable-length sequence packing throughout. The global-attention RoPE base increases from 5×10^5 at 32K to 10^6 in the subsequent phases; the hybrid attention pattern, 4K sliding window, and SWA RoPE base of 10^4 remain fixed.

Optimization across Phases. The mid-training learning rate follows a single cosine decay across all three context-length phases, with no additional warmup. At phase transitions, we transfer model weights and reinitialize the optimizer state while preserving the learning-rate schedule’s progress. Muon momentum is set to 0.9 during mid-training.

3.3.2. Trajectory-Guided Agentic Data Construction

We construct agentic mid-training data by combining real usage trajectories with pre-training documents, building on recent work on scaling terminal-agent trajectory generation in dockerized environments [51]. Trajectories provide realistic task and behavioral priors but have limited coverage, whereas pre-training documents contain broader technical knowledge without an explicit action–observation structure. Our pipeline converts these complementary sources into executable, repository-grounded tasks. It emphasizes reusable component skills that can be instantiated across codebases and composed into longer workflows.

Episode-Guided Knowledge Retrieval. We segment usage trajectories into coherent episodes using rule-based boundary proposals followed by lightweight model-based grouping. Each episode is summarized by its intent, operation, technical context, and observed actions, and the summary serves as a query over the pre-training corpus. A lightweight reranker trained on teacher-model judgments ranks retrieved documents by semantic relevance and task-grounding feasibility. We encode each selected passage as an *intent card* with two components: a repository-agnostic intent core that specifies desired behaviors and constraints, and a binding specification that describes the repository capabilities required to instantiate the intent.

Repository Grounding and Environment Construction. Repository grounding instantiates the binding specification in each intent card. The required capabilities guide repository and code

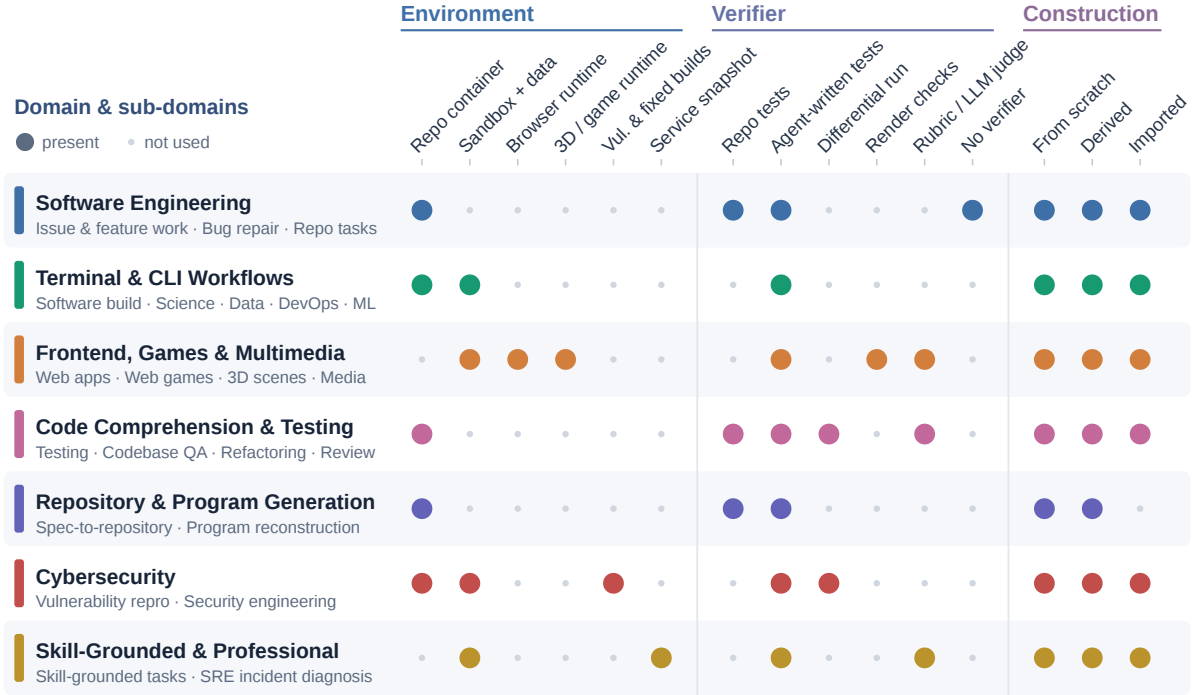


Figure 5. Overview of coding-agent task instances and their environments, verifiers, and construction methods.

search. We filter candidate repositories for licensing and environment compatibility, then verify that they support the intent. A model-assisted binding step maps the intent core to specific files and modules and identifies a concrete task gap; rule-based checks verify that all referenced targets exist. We package the grounded repository instances as containerized environments for task execution and evaluation, producing 1M+ environments in total.

Task Synthesis and Trajectory Collection. Given a grounded intent card, a generator proposes candidate tasks, and a separate reviewer checks their consistency with the intent and repository before the instructions are finalized. Agent rollouts in the prepared environments form a 70B token corpus for mid-training. This procedure transfers broad pre-training knowledge into executable interactions while retaining the behavioral priors of real usage trajectories.

3.4. Evaluation

We probe the substrate of agentic intelligence with two measures during pretraining and midtraining stage. First of all, we compute *bits-per-byte (BPB) on held-out agent trajectories*. Low BPB shows that the model has internalized competent behavior, favoring the actions a strong agent would take, and provides a generation-free fidelity signal sensitive to changes long before task scores move. We also decompose long-term agent behavior into *atomic abilities* and measure how reliably the model executes each one correctly. This fine-grained view reveals specific capability gaps and provides direct guidance for data selection and curriculum design. These two evaluation metrics guide data and curriculum design to improve agentic capabilities.

4. Post-Training

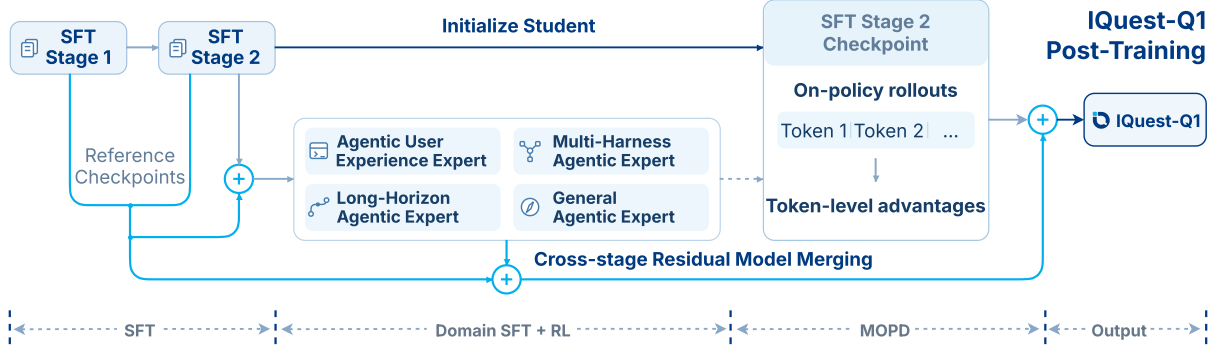


Figure 6. IQuest-Q1 Post-Training Pipeline.

4.1. Large-Scale and Diverse Task Synthesis

We organize the task synthesis framework into three settings: general, reasoning, and coding. Each setting uses diverse data sources and a domain-specific pipeline to generate tasks at scale.

General Agent. We collect APIs and MCP servers from RapidAPI¹ and Smithery², public skills from the web, and workspace files for task synthesis. Guided by the Standard Occupational Classification [52], we cover a broad range of occupations and synthesize high-coverage environments and tasks, which are iteratively refined through a solve-then-refine loop.

Reasoning Agent. We synthesize tasks through a document-grounded pipeline that jointly controls concept coverage, material quality, and task difficulty. The pipeline first expands a broad concept inventory into fine-grained scientific topics, then retrieves and curates documents for each topic, filters the documents before synthesis, and finally converts the retained materials into tasks with reference answers. By covering high-value knowledge through the document collection, the pipeline promotes corresponding coverage in the synthesized prompts.

Coding Agent. We construct executable environments from GitHub repositories using coding-agent harnesses such as Claude Code and Codex, with our model driving environment construction and skills supporting setup. Task queries draw on PRs and issues, repository knowledge graphs and documentation, and reusable patterns extracted from cleaned, de-identified usage logs. Skills capture task structure and common failure modes to guide instance synthesis. Verifiers combine agent-written tests, adapted repository tests, and checks derived from documented behavior. For offline instances, external dependencies are replaced by local fixtures or mocks. An agent-judge loop checks executability, leakage, and consistency among instructions, reference solutions, environments, and verifiers, repairing defective instances and rejecting weak assertions. Reference solutions must pass while no-op solutions must fail. Rubric-based trajectory review further filters cheating and low-quality execution beyond binary success. Repeated agent runs estimate difficulty, and overly easy instances are enriched with additional skills or constraints before revalidation; valid instances exposing model failures are retained.

An overview of coding-agent task instances is shown in Figure 5.

4.2. Supervised Fine-Tuning

In Figure 6, We initialize supervised fine-tuning (SFT) from the mid-trained checkpoint to prepare the policy for reinforcement learning. We optimize the autoregressive cross-entropy

¹<https://rapidapi.com>

²<https://smithery.ai>

loss on assistant responses, including tool-call tokens, while masking user instructions and tool observations from the loss. SFT proceeds in two stages (240B+ token in **Stage 1** and 50B+ token in **Stage 2**), with initial learning rates of 5×10^{-5} and 1.5×10^{-5} , respectively, and cosine learning-rate decay within each stage. Multiple samples are packed into each sequence, with attention masking to prevent cross-sample attention.

4.3. Environment-Centered Reinforcement Learning

We outline an RL design that targets verifiable task completion while accounting for user constraints and execution cost. It draws on configurable-harness training in Kimi K3 [8] and domain-specialized Group Relative Policy Optimization (GRPO) in DeepSeek-V4 [6].

Multi-harness, Multi-domain Episodes. Each episode specifies a task domain, harness, environment, and evaluator. A shared policy operates through different harnesses within a training run, observing public instructions, tools, and execution feedback while reference solutions and hidden checks remain isolated. GRPO compares repeated rollouts of the same task under a fixed harness and evaluator version. Sampling balances domain-harness combinations and retains regression cases.

Rewards and Failure Attribution. Independent verifiers determine terminal correctness. For long trajectories, auxiliary segment-level rewards are restricted to measurable progress, while penalties address unnecessary computation, constraint violations, and reward hacking. Model-generated diagnoses require corroboration from execution records before contributing to these signals. Failures are attributed before optimization: policy-caused errors supply learning signals, environment or harness faults unrelated to policy behavior are masked, and evaluator faults block sample consumption. Token-level updates use recorded rollout probabilities through the existing Token-In, Token-Out (TITO) pipeline. The resulting specialized policies can subsequently be consolidated through MOPD.

4.4. Multi-Teacher On-Policy Distillation

In Figure 6, We initialize the student from SFT Stage 2 and distill four frozen expert models: (1) an *agentic user experience expert* for terminal interaction, tool use, web development, and coding workflows; (2) a *multi-harness agentic expert* for operation across Claude Code, Codex, and OpenCode; (3) a *long-horizon agentic expert* for extended coding and cybersecurity tasks; and (4) a *general agentic expert* for working-agent tasks, web search, multi-turn conversation, and reasoning.

Following MOPD [8, 53], we collect B rollouts from $p_k = p_{\theta_k}$ and select a frozen teacher q_{d_i} for each task x_i according to its domain, $d_i = d(x_i)$. Given the interaction context $h_{i,t}$, the teacher evaluates student-generated tokens to obtain a clipped distillation advantage:

$$\tilde{a}_{i,t} = \text{clip} \left(\log \frac{q_{d_i}(y_{i,t} \mid h_{i,t})}{p_k(y_{i,t} \mid h_{i,t})}, -c, c \right), \quad c > 0. \quad (4)$$

Motivated by reverse-KL distillation [54], we optimize the length-normalized surrogate:

$$L_k(\theta) = -\frac{1}{B} \sum_{i=1}^B \frac{m_i}{T_i} \sum_{t \in \mathcal{A}_i} \tilde{a}_{i,t} \log p_{\theta}(y_{i,t} \mid h_{i,t}), \quad (5)$$

where $\mathcal{A}_i$ contains model-generated token positions, including tool-call tokens, and $T_i = |\mathcal{A}_i|$. Instructions and tool observations provide context but are excluded from the loss. We set $m_i = 0$ for repetitive rollouts and $m_i = 1$ otherwise. Rollouts, contexts, advantages, and masks are fixed during differentiation.

4.5. Stabilized Model Merging

In Figure 6, starting from SFT Stage 1, we interleave training with residual model merging across stages and within expert branches, ultimately incorporating expert and MOPD checkpoints into IQuest-Q1. Each merge combines parameter deltas relative to a designated reference checkpoint using prescribed weights and global ℓ_2 -norm calibration, then adds the resulting update back to the reference parameters.

4.6. Post-Training Infrastructure

Multi-scaffold Agent Runtime. A single policy can be trained on trajectories from multiple agent scaffolds within one run. Each training instance specifies its scaffold, task environment, and verifier. The runtime executes the scaffold and verifies the task in an isolated sandbox while preserving tool orchestration, context management, and interaction flow. We map different API protocols to a unified policy interface to reduce the difficulty of downstream integrations.

Rollout-training Consistency. We implement an end-to-end TITO path at the model-serving boundary. Specifically, we apply a session-level prefix tree to retain token histories shared across turns and branches. A request then combines its environment observations with a recorded token prefix. The corresponding response is returned in the structured format expected by the scaffold. Finally, we retain trajectories paired with rewards and token-level masks from the tree at the end of the sequence.

Asynchronous Distillation and Teacher Execution. We implement MOPD in a fully asynchronous training system. We apply periodic synchronization of student parameters and explicit control of sample staleness, limiting the policy lag introduced by asynchronous execution. We execute teacher forward passes with the training engine on trajectories generated online by the student, producing token-level teacher log probabilities. We use the same execution path for teacher evaluation and student log-probability computation to reduce discrepancies caused by the training engine.

This framework supports independently configured teachers through a common scheduling interface. Teachers reuse the same execution resources through weight switching. We store teacher weights in distributed storage and load them on demand, with only the shards required by the active teacher placed in GPU memory. Weights of the next teacher are prefetched during the current forward pass with overlapping computation. This design supports additional teachers without a proportional increase in resident GPU replicas.

5. Evaluation

5.1. Benchmarks

We evaluate IQuest-Q1 on a comprehensive suite of benchmarks spanning two broad capability axes, covering both coding and general agentic capabilities:

- **Coding Agent:** DeepSWE v1.1 [55, 56], SWE-bench Multilingual [57, 58], SWE-Bench Pro [59], SWE Atlas [60], NL2Repo-Bench [61], ProgramBench [62], CyberGym [63], SkillsBench [64], Terminal-Bench 2.1 [65, 66], Terminal-Bench 3.0 [65, 67], Terminal-Bench 4.0 [65, 68], PaperBench [69], LiveCodeBench v6, IQuest-CLIBench.
- **General Capabilities:** GPQA Diamond [70], Humanity’s Last Exam [71], GDPval [72], Toolathlon [73], Claw-Eval [74], AutomationBench [75], JobBench [76], Agents’ Last Exam (ALE) [77], τ^3 -Banking [78, 79], BrowseComp [80], WideSearch [81], Humanity’s Last Exam(w/tools) [71], HealthBench Professional [82], IMO-AnswerBench-Verified [83], CritPt [84].

5.2. Baselines

We compare IQuest-Q1 with four open-source models and two closed-source models, i.e., GLM-5.3-Flash, DeepSeek-V4-Flash, Hy3, MiniMaxM3, Claude Opus 5, and GPT-5.6 Sol. Agentic scores depend on the execution scaffold, model-harness pairs are treated as the unit of comparison.

5.3. Evaluation Configurations

Agent Harnesses. For agentic coding tasks, we report scores using Claude Code [12] as the harness except for DeepSWE v1.1. We evaluate models with mini-SWE-agent [85] for DeepSWE v1.1.

Reporting Protocol. We attribute each score to the complete model-harness-protocol configuration rather than to the language model alone. This distinction is especially important when comparing Claude Code, mini-SWE-agent, R2E-Gym, and Terminus 2, which expose different tools and execution policies.

Benchmark-specific Metrics. For PaperBench, we report the average Replication Score across papers. For CLI-Bench, we report both the composite Overall Score and pass^k , explicitly stating k and the number of trials. For LiveCodeBench v6, we report code-generation $\text{pass}@1$ on the official `release_v6` split.

5.4. Results

Table 2 summarizes the performance of IQuest-Q1 across coding and general capability benchmarks. The available results show competitive capabilities of IQuest-Q1, with particular strengths in repository generation, knowledge-intensive reasoning, and workflow automation. We compare the reported model-harness configurations and restrict the analysis to benchmarks with available scores.

Coding and Software Engineering. IQuest-Q1 achieves 63.3 on SWE-Bench Pro, exceeding GLM-5.3-Flash, DeepSeek-V4-Flash-0731, Hy3, and MiniMax M3, whose scores range from 52.8 to 59.4, and approaching GPT-5.6 Sol at 64.6. On NL2Repo-Bench, IQuest-Q1 reaches 63.5, outperforming the strongest of these four baselines by 7.2 points and GPT-5.6 Sol by 6.7 points, while remaining below Claude Opus-5 at 75.3. These results IQuest-Q1’s capabilities handling software modification and repository generation. On CyberGym, IQuest-Q1 obtains 81.87, compared with 81.6 for GLM-5.3-Flash, 78.1 for Claude Opus-5, and 83.6 for GPT-5.6 Sol. Its PaperBench Replication Score of 85.9 is close to GLM-5.3-Flash at 86.5 and DeepSeek-V4-Flash-0731 at 85.8, extending this competitive performance to research-code reproduction.

Reasoning and Knowledge. IQuest-Q1 scores 93.1 on GPQA Diamond, 40.1 on HLE without tools, and 57.2 on HLE with tools, exceeding the four baselines GLM-5.3-Flash, DeepSeek-V4-Flash-0731, Hy3, and MiniMax M3 on all three benchmarks. On GPQA Diamond, it is within 0.3 points of Claude Opus-5 and 1.5 points of GPT-5.6 Sol. The HLE gaps are larger: Claude Opus-5 and GPT-5.6 Sol score 54.9 and 49.5 without tools, and 60.9 and 60.2 with tools, respectively. IQuest-Q1 obtains 93.0 on IMO-AnswerBench-Verified, the highest reported score in the table for that benchmark. These results indicate strong performance on knowledge-intensive and mathematical reasoning, while algorithmic coding remains less competitive: its LiveCodeBench v6 score of 86.0 trails GLM-5.3-Flash at 88.0 and DeepSeek-V4-Flash-0731 at 95.4.

General Agentic Capabilities. IQuest-Q1 achieves 52.7 on AutomationBench, the highest score among all seven systems in the table, exceeding Claude Opus-5 by 2.4 points and GPT-5.6 Sol by 6.9 points. It also attains the highest reported Claw-Eval score at 76.7. On JobBench and Agents’

Table 2. Performance across coding and general capability benchmarks. Scores are reported at the highest available reasoning effort for each model. DS denotes DeepSeek. [†]CyberGym uses Claude Opus 4.8 as a fallback. Results with * are from our own testing.

Benchmark	IQuest-Q1	GLM-5.3 Flash	DS-V4 Flash-0731	Tencent Hy3	MiniMax M3	Claude Opus-5	GPT-5.6 Sol
Coding Capabilities							
DeepSWE v1.1	69.0	63.4	53.1*	28.0	-	73.7	73.0
SWE-bench Multilingual	80.3	79.7*	79.2*	74.0*	75.0*	85.8*	74.1*
SWE-Bench Pro	63.5	59.4*	57.3*	57.9	59.0	79.2	64.6
SWE Atlas	45.4	43.7*	46.1*	30.8	36.6*	63.2	46.0
NL2Repo-Bench	63.0	56.3	54.2	45.6	42.1	75.3	56.8
ProgramBench	61.5	58.5*	62.1*	47.4*	33.8*	82.3*	77.6*
CyberGym	84.5	81.6*	76.7	51.8	40.3*	78.1* [†]	83.6
SkillsBench	74.1	56.2*	71.7*	63.9*	54.2*	76.2*	71.5*
Terminal-Bench 2.1	83.2	84.3	82.7	70.8*	65.2*	85.4	88.3
Terminal-Bench 3.0	17.9	31.8*	7.6	3.2*	1.8*	42.7	34.6
Terminal-Bench 4.0	23.8	27.0*	6.0	4.8*	1.6*	51.8	37.3
IQuest-CLIBench	53.7	49.5*	51.5*	36.6*	30.7*	58.2*	58.5*
PaperBench	86.0	86.5*	85.8*	68.4*	67.7*	91.8*	90.5
LiveCodeBench v6	89.7	88.0*	90.6	91.0	81.1*	-	-
General Capabilities							
GPQA Diamond	93.3	92.2	89.9	90.9	92.9	93.4	94.6
Humanity’s Last Exam	39.2	39.9	38.6	33.5	39.0	54.9	49.5
Humanity’s Last Exam (w / tools)	56.5	55.3	51.5	53.2	41.2*	60.9	60.2
CritPt	18.3	15.4	16.6	4.9	3.7	29.1	32.3
IMOAnswerBench-Verified	93.0	75.3*	92.8*	92.8*	79.6*	-	-
GDPval v2.1	1530	1641	1427	1045	1230	1708	1588
Toolathlon	75.0	78.4	70.3	56.2	53.7*	80.6	74.9
Claw-Eval	77.2	74.2*	68.8*	63.5*	69.8*	79.6*	70.2*
AutomationBench	52.7	48.8	37.7	16.1	19.7*	50.3	45.8
APEX-Agents 1.1	50.9	52.8	36.9*	-	-	65.8	51.4
JobBench	55.7	49.7*	50.0*	34.6	40.1*	65.7	45.4
Agents’ Last Exam	29.6	26.3	25.2	17.1	9.1*	32.2	30.6
τ^3 -Banking	51.8	47.2	39.4	22.9	15.3	44.7	44.3
BrowseComp	88.5	-	78.0	84.2	83.5	90.8	90.4
WideSearch	81.8	-	80.5	81.9	73.5	84.0	86.3
HealthBench Professional	33.4	47.1*	26.4*	29.8*	24.7*	59.8	60.5

Last Exam, scores of 55.5 and 29.4 exceed all four baselines, although Claude Opus-5 remains ahead at 65.7 and 31.6, respectively. The GDPval v2.1 score of 1530 lies between GLM-5.3-Flash at 1641 and DeepSeek-V4-Flash-0731 at 1427, and below GPT-5.6 Sol at 1588. Tool-use results reveal further room for improvement: IQuest-Q1 scores 70.0 on Toolathlon, below both Claude Opus-5 and GPT-5.6 Sol. Together, these results demonstrate broad utility across general agent workflows, with remaining gaps in reliable tool orchestration and demanding terminal tasks.

6. Capabilities with CLI Agents

6.1. Human-on-the-Loop Recursive Self-improvement

During the development of IQuest-Q1, we established a human-on-the-loop RSI loop that enables IQuest-Q1 to build itself. This human-on-the-loop RSI enables the current model to contribute to the development of its successor under human supervision [86, 87]. As illustrated in Figure 7, an RSI agent carries out this process based on the latest checkpoint from the ongoing IQuest-Q1 development cycle and a development harness. The model is both the object of improvement and the development executor [88, 89]. The harness organizes this model-development process with tools, workflows, skills, reusable procedures, and experimental

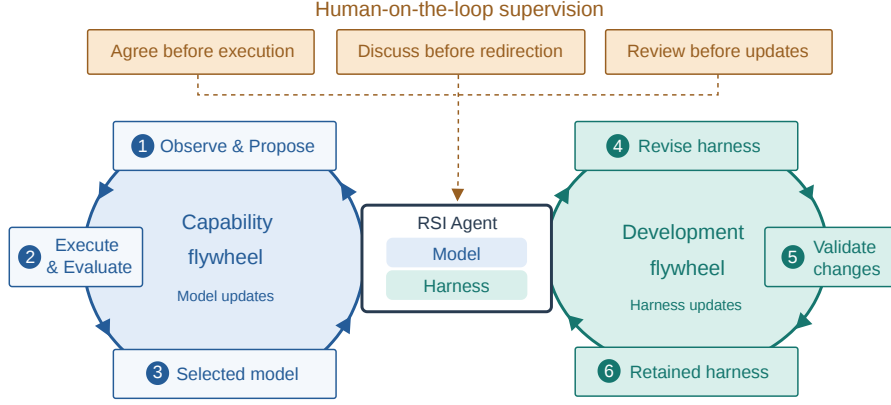


Figure 7. Human-on-the-loop RSI system design.

records; accepted training assets and harness updates persist into later iterations [90, 91].

6.1.1. System Design

Two design principles organize the RSI process: (1) *nested capability and development flywheels*, which improve both the model and the process used to develop it [92]; and (2) *human-on-the-loop supervision*, with human intervention at key decision points, including experiment planning, costly execution, and update adoption.

Nested Capability and Development Flywheels. We organize RSI into two nested and reciprocally coupled improvement loops, one improving model capabilities and the other evolving the development harness:

(1) *Capability flywheel.* The capability flywheel improves *model capabilities* by identifying capability gaps and proposing training or methodological improvements to address them. For each proposal, the agent runs experiments to determine whether the resulting model meets the intended objective without performance regressions. The resulting model, if validated, becomes the basis for the next iteration, enabling further capability improvements.

(2) *Development flywheel.* The development flywheel improves the model development process by evolving the *harness* through which the RSI agent works [89]. The agent uses evidence from successful and unsuccessful iterations to identify failures and inefficiencies, and develops reusable code and procedures for auditing, data cleaning, and validation. Validated improvements persist into subsequent iterations, improving the reliability and efficiency of model development [93].

Human-on-the-Loop Supervision. RSI couples uncertain decisions with costly experiments: an ineffective decision can waste training resources, while adopting a degraded model can propagate errors across iterations. We therefore maintain human supervision throughout the RSI process to mitigate these risks [86]. Specifically, three rules govern this process: (1) *Agree before execution.* Before executing large-scale experiments, humans and the agent agree on the experimental objective, setup, and required resources [88]. (2) *Discuss before redirection.* When unexpected findings challenge the initial hypothesis, humans and the agent discuss the implications before deciding whether to continue or revise the experiment [94]. (3) *Review before adoption.* Humans review model and harness updates to prevent errors from accumulating across iterations [95].

Infrastructure Support. We integrate our internal model-development infrastructure into a unified agent workspace that connects data pipelines, algorithm implementations, training

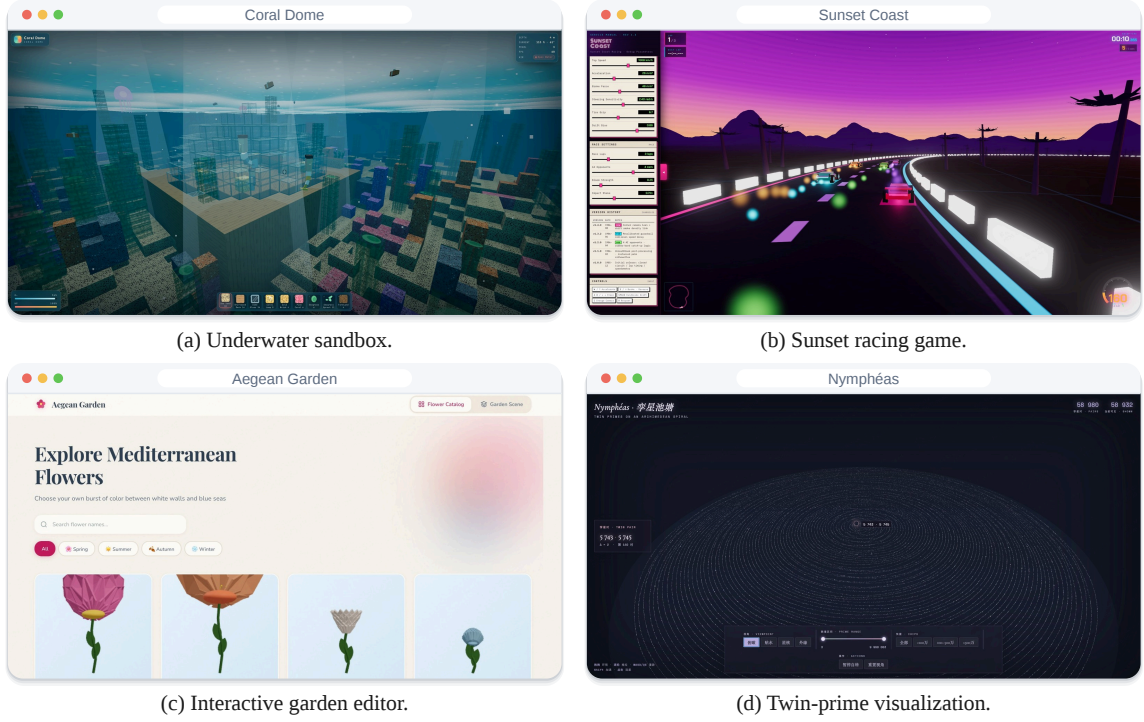


Figure 8. Representative interactive frontend applications generated by IQuest-Q1 from natural-language specifications, covering game worlds, scene editing, and scientific exploration.

workflows, and evaluation benchmarks with container image services, sandboxed execution environments, and compute resources. Through these interfaces, the agent can construct data, implement training methods, launch approved experiments, and inspect their results.

6.1.2. A Minimal RSI Loop under Human Supervision

We demonstrate a representative RSI iteration in which the agent identifies a capability gap, designs and implements a solution, and evaluates the resulting model under human supervision. The iteration targets the model’s ability to adapt to different agent scaffolds and their tool call schema. While the model performs well with familiar tool definitions, it can fail on unfamiliar schema; in one long-horizon task, it produced 35 consecutive tool-call errors because the tool names and parameters did not match the scaffold.

Trace Tool-call Failures and Identify Data Inconsistencies. The agent based on IQuest-Q1 traces the failure and inspects 6,054 tool calls across 200 training records, identifying a strong bias toward a single schema and inconsistencies between tool definitions and call parameters introduced during data preprocessing. It proposes cleaning the inconsistent records and augmenting the training data with mixed tool call schemas.

Implement Mixed-schema Data Augmentation and Model Training. The agent implements the data cleaning and augmentation pipeline and prepares the experiment configuration. The training run uses 2.35B tokens and 938.7 GPU hours. The resulting model checkpoint is evaluated on held-out schemas and familiar schemas, while checking for performance regressions.

Model Selection and Harness Update. The resulting checkpoint improves PaperBench-CodeDev from 61.04% to 68.28%, an absolute improvement of 7.24%. Following human review and approval, the agent selects the improved checkpoint, integrates the validated data cleaning and

tool call schema augmentation into the harness. The validated model and harness are then carried forward to the next iteration.

During this iteration, humans expert supervise key decision points: they review the proposed objective, setup, and required resources before large-scale training, authorize the training run, and review the resulting model checkpoint and harness changes before adoption. This keeps the agent responsible for driving the development loop while humans retain control over costly experiments and changes carried forward to subsequent iterations.

6.2. Cyber Security

Beyond software engineering and coding, we observe that these improvements naturally generalize to defensive (blue-team) cybersecurity tasks. IQuest-Q1 achieves strong performance on CyberGym [63], a benchmark for real-world vulnerability analysis over large open-source code-bases. We further validate this capability in real-world settings. During a three-day deployment, IQuest-Q1, equipped with Codex agents and expert-designed cybersecurity skills, produced 30 responsibly submitted vulnerability reports, all reviewed by human cybersecurity experts. Of these, 13 findings were independently validated, with records subsequently published by established security organizations and databases, including GitHub Security Advisories (GHSA) and NVD. Together, these results suggest that IQuest-Q1’s vulnerability analysis capabilities extend beyond fixed benchmark settings to real-world software targets.

6.3. Interactive Frontend Generation

IQuest-Q1 also generates diverse interactive frontend applications given user queries, combining visual design, scene construction, and interaction logic in browser-runnable HTML, CSS, and JavaScript. These tasks provide a concrete application of its command-line interface (CLI) coding capabilities, connecting implementation in an agent workspace to an interactive artifact that users can directly explore.

We provide in Figure 8 examples from domains including games, creative tools, and scientific visualization. For example, the underwater sandbox combines an explorable world with building and mining, while the sunset racing game couples vehicle control with camera movement and race-state feedback. Another showcase with the garden editor links flower parameters and planting actions to a 3D scene. Together, they strongly suggest how IQuest-Q1 coordinates rendering, user input, and application state to create interactive web experiences with distinct visual styles.

Conclusion

We present IQuest-Q1, an open-source agentic foundation model supporting general and coding agents. The model combines a stable, efficient sparse MoE architecture with an end-to-end training pipeline: broad pre-training with increasing emphasis on code and STEM; mid-training with diverse agent trajectories and context extension to 512K tokens; and post-training grounded in task and environment synthesis, supervised fine-tuning, multi-scaffold reinforcement learning, and multi-teacher on-policy distillation.

The resulting model achieves competitive performance across several coding-agent benchmarks and supports practical CLI interaction. Beyond the model itself, our Human-on-the-Loop framework allows the current checkpoint to contribute to model development under human guidance and review. Together, the architecture, training pipeline, and human-supervised

development process provide a solid foundation for continued model development and a practical starting point for studying recursive self-improvement.

7. Contributions and Acknowledgements

The authors of this paper are listed in order as follows:

Core Contributors: Jian Yang, Tianyu Zheng, Wei Zhang, Ji Chen, Qi Yun, Fang Qiu, Shukai Liu, Linzheng Chai, Hongyuan Tao, Feng Chang, Wenrui Cai, Chengran Yang, Hongxin Li, Yan Xing, Chuan Shi, Chuan Yu, Chen Mu, Shi Qing, Yizhi Li, Ling Shi, Ming Jue, Cheng Si, Yu Zhu, Feng Mu, Yun Xing, Gan Bing, Tao Zhen, Xiao Qing, Lan Yun, Wei Liu, Heng Shuo, Yun Qing, Jujie He, Yuan Tao, Jinxing Zhang, Yi Xing, Mo Jing, Ming Yang, Yiran Xing, Bo Dan, Ge Huan.

Contributors Xudong Ma, Yaxin Du, Jun Ce, Wenlong Miao, Xiaocheng Li, Yang Tian, Jiajun Wu, Xiyuan Yang, Pei Wang, Weihang Peng, Weicheng Gu, Zhizhao Luo, Shinan Zhang, Zixi Zhao, Qixiang Yin, Yibo Tan, Xin Li, Kaixiang Pan, Guochang Li, Kun Zeng, Huanjin Yao, Ziyi Wang, Fanglin Xu, Zhiwei Yu, Hao Tang, Qing Chen, Qi Nai, Jingda Xu, Yue Yuan, Cheng Feng, Tang Chuan, Yun Cang, Jiyu Wang, Xin Yi, Xu Zhu, Qi Wu, Qing Lian, Cai Yun, Yuan Ze, Zichen Wang, Bo Luo, Ji De, Hei Yu, Guo Ting, Tian Yu, Yun Shu, Xing Ti, Shengjie Fang, Du Ruo, Lin Yuan, Chang Shu, Yun Qing, Jin Jie, Lan Chuan, Jing Nan, Yi Qing, Zhi Xin, Qing Feng, Ling Yang, Cheng Zi, Wei Yin, Bei Ke, Ling Yi, Jin Ran, Ying Chun, Shi Qi, Zhi Qiu, Fan Wang, Ju Zi, Yi Song, Zhi Shui, Lin Yun, Cao Zhi, Dong Yang, Dao Yuan, Shan Jin, Bohan Yang, Hong Yan, Qi Jiu, Xing Yue, Yi Yan, Yi Xin, Yue Feng, Hai Tang, Fang Zhou, Zhao Ye, Hailong Zhang, Dong Ning, Qing Zhou, Qing Han, Linjia Han, Yun Sen, Xiang Yang, Zhi Yi, Qiu Liang, Xiang Zhi, Tong Ya, San Feng, Shan Que, Shu Bing, San Qi, Guan Lan, Lin Cang, Bi Ge, Yan Yue, Chen Da, Lin Yao, Cheng Huai, Le Cheng, Yi Zhi, Yu Mao, Zhu Yi, Huai Min, Yun Xiao, Lin Zhu, Guan Yue, Bo Yue, Qi Yao, Jing Feng, Gui Zhou, Chi Yi, Jian Qiu, Kai Bo, Cheng Guang, Hai Tao, Yi Bin, Zhao Han, Ru Mi, Ling Rui, Qing Yue, An Ka, Da Shu, Sheng Yan, Zhi Bai, Ying Jie, Jia Yi, Jian Wu, Xiao Chun, Lian Qiao, Xin Ye, Xiao Yi, Zhi Hui, Mi Yao, Ming Yi, Si Yan, Xue Li, Ruochen Wang, Qi Dian, Shi Yu, Bi Luo, Zhao Heng, Zhen Yan, Jin Song, Xu Heng, Jia Mu, Wan Feng, Chen Hao, Wu Ji, Qin Zheng, Shi An, Ruo Gu, Fei Yu, Hao Lin, Ju Heng, Wentao Zuo, Mu Jie, Jing Chen, Da Bai, Shujiao Li, Ting Yun, Yuhua Yang, Jing Yu, Li Ming, Wu Yuan, Long Zhao, You Xi, Xing Chen, Wei Cheng, Ying Liu, Yu Wu, Xiao Yu, Zhi Qian, Chang Qing, Ye Zi, Yi Chuan, Jun Ru, Yi Yi, Yi Qi, Ran Su, Ze Tu, Xiao Guang, Wang Chuan, Chun Ji, Hua Jiu, Yuhang Xun, Qing Lang, Jiao Yang, Jiu Qi, Da Dou, Ruo Yan, Chen Shu, San Mu, Bo Pai, Li Ke, Wan Yun, Ming Qian, Zhao Zhao, Shu Heng, Quan Jiu, Jiu Yue, Xiang Yu, Mao Lv, Shan Tian, Congyu Wang, Mo Zha, Ming Chuan, Huo Xing, Xi Chen, Xiao Chen, Xin Lan, Ai Si, Jiejun Tan, Ziyu Zhao, Haowen Wang, Chunjiang Ge, Junhang Cheng, Jing Yang, Huatong Song, Wei Yuan, Jian Shen, Danyang Li, Kai Lv, Zhaoyi Li, Zhixin Xie, Yuxuan Zhang, Yinda Chen, Yachao Wang, Zhiyu Chen

Leadership and Senior Advisory Committee: Ran Tao, Mingjie Tang, Chuan Hao, Peihao Wu, Jianzhou Wang, Nan Jiang, Yuan Zhi.

Corresponding Authors: Bryan Dai.

References

- 1 Jian Yang, Xianglong Liu, Weifeng Lv, Ken Deng, Shawn Guo, Lin Jing, Yizhi Li, Shark Liu, Xianzhen Luo, Yuyu Luo, et al. From code foundation models to agents and applications: A comprehensive survey and practical guide to code intelligence. *arXiv preprint arXiv:2511.18538*, 2025.
- 2 Jian Yang, Wei Zhang, Jiajun Wu, Junhang Cheng, Shawn Guo, Haowen Wang, Weicheng Gu, Yaxin Du, Joseph Li, Fanglin Xu, Yizhi Li, Lin Jing, Yuanbo Wang, Yuhan Gao, Ruihao

- Gong, Chuan Hao, Ran Tao, Aishan Liu, Tuney Zheng, Ganqu Cui, Zhoujun Li, Mingjie Tang, Chenghua Lin, Wayne Xin Zhao, Xianglong Liu, Ming Zhou, Bryan Dai, and Weifeng Lv. Incoder-32b: Code foundation model for industrial scenarios, 2026. URL <https://arxiv.org/abs/2603.16790>.
- 3 Jiajun Wu, Jian Yang, Wei Zhang, Lin Jing, Yuqing Ma, Ensheng Shi, Yuchi Ma, Zhoujun Li, and Xianglong Liu. Ucoder: Unsupervised code generation by internal probing of large language models, 2025. URL <https://arxiv.org/abs/2512.17385>.
 - 4 OpenAI. GPT-5.6 system card, July 2026. URL <https://deploymentsafety.openai.com/gpt-5-6>. Published July 9, 2026; accessed September 28, 2026.
 - 5 Anthropic. Introducing Claude Opus 5, July 2026. URL <https://www.anthropic.com/news/claude-opus-5>. Published July 24, 2026; accessed September 28, 2026.
 - 6 DeepSeek-AI. DeepSeek-V4: Towards highly efficient million-token context intelligence, 2026. URL <https://arxiv.org/abs/2606.19348>. Technical report, version 1.
 - 7 Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chenghua Huang, Chengxing Xie, et al. Glm-5: from vibe coding to agentic engineering. *arXiv preprint arXiv:2602.15763*, 2026.
 - 8 Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, M. C., Jianfeng Cai, Xinyuan Cai, Peizhou Cao, Yuxuan Cao, Ziwei Chai, Y. Charles, H. S. Che, Guanduo Chen, Guangyu Chen, Guanzheng Chen, Huarong Chen, Jia Chen, Jianlong Chen, Jun Chen, Kexin Chen, Peng Chen, Ruijie Chen, Wentao Chen, Xin Chen, Yang Chen, Yanru Chen, Yifei Chen, Yingjiang Chen, Yuankun Chen, Yujie Chen, Yutian Chen, Zhirong Chen, Dazhi Cheng, Yean Cheng, Jialei Cui, Jingbing Cui, Anqi Dai, Jiaqi Deng, Hao Ding, Rui Ding, Shaofeng Ding, Mengfan Dong, Mengnan Dong, Yuhao Dong, Yuxin Dong, Angang Du, Chenzhuang Du, Dikang Du, Jusen Du, Yulun Du, Yu Fan, Jing Feng, Qiulin Feng, Yichen Feng, Kelin Fu, Qiang Fu, Fuxuan Gao, Hongcheng Gao, Jingyue Gao, Tong Gao, Weijia Gao, Shangyi Geng, Jie Gong, Linhu Gong, Shengao Gong, Xiaochen Gong, Qizheng Gu, Yicheng Gu, Shuhao Guan, Haiqing Guo, Shiqi Guo, Xiang Guo, Zhengyan Guo, Beixi Hao, Wenxin Hao, Xiaoru Hao, Dailan He, Haotian He, Lehan He, Qi He, Weiran He, Xinran He, Xinyi He, Yibo He, Yunjia He, Chao Hong, Tiange Hong, Hao Hu, Jiayi Hu, Ruikun Hu, Weiming Hu, Yangyang Hu, Zhenxing Hu, Liang Hua, Jinbin Huang, Ke Huang, Ruiyuan Huang, Siying Huang, Weixiao Huang, Yan Huang, Zhengjie Huang, Zhiqi Huang, Yulong Hui, Chaobo Jia, Yutong Jiang, Zhejun Jiang, Zuoyou Jiang, Wenyi Jin, Xinyi Jin, Yu Jing, Huanjun Kong, Guokun Lai, Aidi Li, Cheng Li, Chengyuan Li, Cong Li, Fang Li, Guanyu Li, Haoyang Li, Jia Li, Junxiong Li, Lei Li, Letian Li, Lincan Li, Weihong Li, Wentao Li, Xintong Li, Yang Li, Yishen Li, Yiwei Li, Yuxiao Li, Zhaowei Li, Zhaoxi Li, Zheming Li, Zhengxiao Li, Zhiyuan Li, Jiawei Lin, Xiaohan Lin, Yibo Lin, Zichao Lin, Ziyan Lin, Bill Liu, Boxiao Liu, Chuan Liu, Liang Liu, Shaowei Liu, Shudong Liu, Shuran Liu, Tianwei Liu, Weizhou Liu, Yangyang Liu, Yanming Liu, Yibo Liu, Yipeng Liu, Zhengying Liu, Zhiheng Liu, Enzhe Lu, Haoyu Lu, Linqiang Lu, Tingzhan Lu, Zhiyuan Lu, Aotian Luo, G. Luo, Junyu Luo, Yifan Luo, B. Lyu, Wenzhou Lyu, Shaoguang Mao, Yuan Mei, Xin Men, Mingqing Ni, Yixuan Niu, Siyuan Pan, Shujun Peng, Zhangyang Qi, Ruoyu Qin, ZeChao Qin, Zeyu Qin, Haiquan Qiu, Jianxin Qiu, Jiezhong Qiu, Bowen Qu, Yuhao Qu, Zeyu Shang, Youbo Shao, Han Shen, Jincheng Shi, Juanfeng Shi, Lidong Shi, Shengyuan Shi, Wingchun Siu, Pengwei Song, Xiaoxi Song, Jianlin Su, Yunfeng Su, Zhaochen Su, Lin Sui, Jingsong Sun, Junyao Sun, Shaoning Sun, Shuzhe Sun, Tongyu Sun, Yujun Sun, Yunpeng Tai, Chuning Tang, Heyi Tang, Sirui Tang, Zecheng Tang, Chaoran Tian, Rongpeng Tian, Yu Tian, Wei Tu, Chensi Wang, Chuang Wang, Chunjie Wang, Dinglu

- Wang, Feng Wang, Hailong Wang, Haiming Wang, Hao Wang, Hao Wang, Huaqing Wang, Hui Wang, Jiayi Wang, Jinglong Wang, Jinhong Wang, Jiuzheng Wang, Linian Wang, Shaobo Wang, Shenzhi Wang, Shuyi Wang, Si Wang, Siyuan Wang, Tianfu Wang, Wenjue Wang, Xingran Wang, Xinmei Wang, Xinyuan Wang, Xusheng Wang, Yalin Wang, Yangkun Wang, Yao Wang, Yaoyu Wang, Yejie Wang, Yiqin Wang, Yucheng Wang, Yuzhi Wang, Zhaoji Wang, Zhaowei Wang, Zhengtao Wang, Zhenhao Wang, Zhongsheng Wang, Zifan Wang, Chu Wei, Ming Wei, Shouxin Wei, Zichen Wen, Fan Wu, Haoning Wu, Rucong Wu, Wenhao Wu, Xiaoxue Wu, Yingcong Wu, Yongqi Wu, Yuxin Wu, Zijian Wu, Xinglang Xian, Chenxuan Xiang, Yuye Xiang, Bocheng Xiao, Chenjun Xiao, Xin Xiao, Jin Xie, Xiaotong Xie, Yifeng Xie, Zhe Xie, Bowei Xing, Yiming Xiong, Baosheng Xu, Boyu Xu, Jiale Xu, Jianfan Xu, Jing Xu, Jinjing Xu, L. H. Xu, Qingtao Xu, Shuyao Xu, Suting Xu, Tiantian Xu, Tianxiang Xu, Weixin Xu, Xinran Xu, Yangchuan Xu, Ye Xu, Yueni Xu, Ziyao Xu, Haonan Xue, Junjie Yan, Yaoyao Yan, Fan Yang, Guangyao Yang, Hao Yang, Junwei Yang, Ruoyu Yang, Wenjie Yang, Xiaofei Yang, Xinyu Yang, Yi Yang, Yiling Yang, Ying Yang, Yuchen Yang, Zhen Yang, Zhilin Yang, Zian Yang, Zuhao Yang, Haotian Yao, Dan Ye, Haoran Ye, Wenjie Ye, Zhanbo Ye, Bohong Yin, Haoxiang Yin, Xietong Yin, Chengzhen Yu, Haozhen Yu, Longhui Yu, Shengnan Yu, Shuying Yu, Tianxiang Yu, Enming Yuan, Mengjie Yuan, Tongtian Yue, Wei Yue, Yang Yue, Dunyuan Zha, Haobing Zhan, B. H. Zhang, Dehao Zhang, Fei Zhang, Hao Zhang, Haoyuan Zhang, Huanyu Zhang, Jiawei Zhang, Jiaxuan Zhang, Jin Zhang, Kaiyi Zhang, Miaozhen Zhang, Puqi Zhang, Qinglei Zhang, Rong Zhang, Rui Zhang, Shaoshuai Zhang, Shiyi Zhang, Xiaobin Zhang, Xiaoyun Zhang, Y. Zhang, Yangkun Zhang, Ye Zhang, Yichi Zhang, Yikun Zhang, Yizhi Zhang, Yongting Zhang, Yu Zhang, Yutao Zhang, Yutong Zhang, Zheng Zhang, Zijiang Zhang, Bin Zhao, Chenguang Zhao, Feifan Zhao, Jinglun Zhao, Jinxiang Zhao, Shuai Zhao, Wenshuo Zhao, Xiangyu Zhao, Xuanle Zhao, Yikai Zhao, Zijia Zhao, Haozhi Zheng, Huabin Zheng, Ruihan Zheng, Shaojie Zheng, Tengyang Zheng, Haofeng Zhong, Lei Zhong, Longguang Zhong, M. Zhou, Qianqiang Zhou, Runjie Zhou, Ruozhang Zhou, Xinyu Zhou, Yiqiao Zhou, Zaida Zhou, Jinguo Zhu, Liya Zhu, Xinhao Zhu, Yangjunfeng Zhu, Yuxuan Zhu, Zhen Zhu, Chen Zhuang, Weiyu Zhuang, and Xinxing Zu. Kimi k3: Open frontier intelligence, 2026. URL <https://arxiv.org/abs/2607.24653>.
- 9 Jian Yang, Wei Zhang, Shawn Guo, Zhengmao Ye, Lin Jing, Shark Liu, Yizhi Li, Jiajun Wu, Cening Liu, X Ma, et al. Iquest-coder-v1 technical report. *arXiv preprint arXiv:2603.16733*, 2026.
 - 10 Jian Yang, Wei Zhang, Jiajun Wu, Junhang Cheng, Tuney Zheng, Fanglin Xu, Weicheng Gu, Lin Jing, Yaxin Du, Joseph Li, Yizhi Li, Yan Xing, Chuan Hao, Ran Tao, Ruihao Gong, Aishan Liu, Zhoujun Li, Mingjie Tang, Chenghua Lin, Siheng Chen, Wayne Xin Zhao, Xianglong Liu, Ming Zhou, Bryan Dai, and Weifeng Lv. Incoder-32b-thinking: Industrial code world model for thinking, 2026. URL <https://arxiv.org/abs/2604.03144>.
 - 11 OpenAI. Introducing upgrades to codex: Gpt-5-codex. <https://openai.com/index/introducing-upgrades-to-codex/>, 2025.
 - 12 Anthropic. Claude Code overview, 2025. URL <https://code.claude.com/docs/en/overview>. Accessed: 2026-09-27.
 - 13 Zhijian Zhuo, Yutao Zeng, Ya Wang, Sijun Zhang, Xiaoqing Li, Jian Yang, Jinwen Ma, et al. Hybridnorm: Towards stable and efficient transformer training via hybrid normalization. *Advances in Neural Information Processing Systems*, 38:25106–25147, 2026.
 - 14 WeChat AI Team. Building effective sparse moe models with moderate resources, 2026. URL

<https://welm.weixin.qq.com/posts/building-effective-sparse-moe-models-with-moderate-resources/>.

- 15 Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, Yanru Chen, Huabin Zheng, Yibo Liu, Shaowei Liu, Bohong Yin, Weiran He, Han Zhu, Yuzhi Wang, Jianzhou Wang, Mengnan Dong, Zheng Zhang, Yongsheng Kang, Hao Zhang, Xinran Xu, Yutao Zhang, Yuxin Wu, Xinyu Zhou, and Zhilin Yang. Muon is scalable for LLM training. *arXiv preprint arXiv:2502.16982*, 2025.
- 16 Greg Yang, Edward J. Hu, Igor Babuschkin, Szymon Sidor, Xiaodong Liu, David Farhi, Nick Ryder, Jakub Pachocki, Weizhu Chen, and Jianfeng Gao. Tensor programs V: Tuning large neural networks via zero-shot hyperparameter transfer. In *Advances in Neural Information Processing Systems*, volume 34, pages 17084–17097, 2021. URL <https://proceedings.neurips.cc/paper/2021/hash/8df7c2e3c3c3be098ef7b382bd2c37ba-Abstract.html>.
- 17 Shukai Liu, Bo Jiang, Jian Yang, Yizhi Li, Jinyang Guo, Xianglong Liu, and Bryan Dai. Context as a tool: Context management for long-horizon SWE-agents. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens, editors, *Findings of the Association for Computational Linguistics: ACL 2026*, pages 20604–20617, San Diego, California, United States, July 2026. Association for Computational Linguistics. ISBN 979-8-89176-395-1. doi: 10.18653/v1/2026.findings-acl.1032. URL <https://aclanthology.org/2026.findings-acl.1032/>.
- 18 Shukai Liu, Linzheng Chai, Jian Yang, Jiajun Shi, He Zhu, Liran Wang, Jin Ke, Wei Zhang, Hualei Zhu, Shuyue Guo, Tao Sun, Jiaheng Liu, Yunlong Duan, Yu Hao, Liqun Yang, Guanglin Niu, Ge Zhang, and Zhoujun Li. MdEval: Massively multilingual code debugging. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens, editors, *Findings of the Association for Computational Linguistics: ACL 2026*, pages 20780–20797, San Diego, California, United States, July 2026. Association for Computational Linguistics. ISBN 979-8-89176-395-1. doi: 10.18653/v1/2026.findings-acl.1041. URL <https://aclanthology.org/2026.findings-acl.1041/>.
- 19 Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NIPS*, pages 5998–6008, 2017.
- 20 William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23 (120):1–39, 2022. URL <https://jmlr.org/papers/v23/21-0998.html>.
- 21 Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4895–4901. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.emnlp-main.298. URL <https://aclanthology.org/2023.emnlp-main.298/>.
- 22 Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L  lio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. Mistral 7b, 2023. URL <https://arxiv.org/abs/2310.06825>.
- 23 Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K Arora, Yu Bai, Bowen Baker, Haiming Bao, et al. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*, 2025.

- 24 Alex Henry, Prudhvi Raj Dachapally, Shubham Shantaram Pawar, and Yuxuan Chen. Query-key normalization for transformers. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4246–4253. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.findings-emnlp.379. URL <https://aclanthology.org/2020.findings-emnlp.379/>.
- 25 Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024. doi: 10.1016/j.neucom.2023.127063. URL <https://doi.org/10.1016/j.neucom.2023.127063>.
- 26 Róbert Csordás, Christopher D Manning, and Christopher Potts. Do language models use their depth efficiently? In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://arxiv.org/abs/2505.13898>.
- 27 Avi Schwarzschild, Arjun Gupta, Amin Ghiasi, Micah Goldblum, and Tom Goldstein. The uncanny similarity of recurrence and depth. In *International Conference on Learning Representations*, 2022. URL <https://arxiv.org/abs/2102.11011>.
- 28 Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tieyan Liu. On layer normalization in the transformer architecture. In *International conference on machine learning*, pages 10524–10533. PMLR, 2020.
- 29 Wenfang Sun, Xinyuan Song, Pengxiang Li, Lu Yin, Yefeng Zheng, and Shiwei Liu. The curse of depth in large language models. *Advances in Neural Information Processing Systems*, 38:163104–163136, 2026.
- 30 Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL <https://proceedings.neurips.cc/paper/2019/hash/1e8a19426224ca89e83cef47f1e7f53b-Abstract.html>.
- 31 Ming Ding, Zhuoyi Yang, Wenyi Hong, Wendi Zheng, Chang Zhou, Da Yin, Junyang Lin, Xu Zou, Zhou Shao, Hongxia Yang, and Jie Tang. CogView: Mastering text-to-image generation via transformers. In *Advances in Neural Information Processing Systems*, volume 34, pages 19822–19835. Curran Associates, Inc., 2021. URL <https://proceedings.neurips.cc/paper/2021/hash/a4d92e2cd541fca87e4620aba658316d-Abstract.html>.
- 32 Greg Yang, Dingli Yu, Chen Zhu, and Soufiane Hayou. Tensor programs VI: Feature learning in infinite depth neural networks. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/f2ce452a4a7352feb7a9fb885267ff24-Abstract-Conference.html.
- 33 Nolan Dey, Bin Claire Zhang, Lorenzo Noci, Mufan Li, Blake Bordelon, Shane Bergsma, Cengiz Pehlevan, Boris Hanin, and Joel Hestness. Don’t be lazy: CompleteP enables compute-efficient deep transformers. In *Advances in Neural Information Processing Systems*, volume 38, pages 137707–137739, 2025. doi: 10.52202/085713-4597. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/c917d8b9e01427f3184d80ade22f4d1f-Abstract-Conference.html.
- 34 Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Roziere, David Lopez-Paz, and Gabriel Synnaeve. Better & faster large language models via multi-token prediction. In *Forty-first International Conference on Machine Learning*, 2024. URL <https://arxiv.org/abs/2404.19737>.
- 35 DeepSeek-AI. Deepseek-v3 technical report, 2024. URL <https://arxiv.org/abs/2412.19437>.

- 36 Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle-3: Scaling up inference acceleration of large language models via training-time test. *Advances in Neural Information Processing Systems*, 38:136737–136756, 2026.
- 37 Yuxuan Cai, Xiaozhuan Liang, Xinghua Wang, Jin Ma, Haijin Liang, Jinwen Luo, Xinyu Zuo, Lisheng Duan, Yuyang Yin, and Xi Chen. Fastmtp: Accelerating llm inference with enhanced multi-token prediction. *arXiv preprint arXiv:2509.18362*, 2025.
- 38 An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- 39 Sam McCandlish, Jared Kaplan, Dario Amodei, and OpenAI Dota Team. An empirical model of large-batch training. *arXiv preprint arXiv:1812.06162*, 2018.
- 40 Will Merrill, Shane Arora, Dirk Groeneveld, and Hanna Hajishirzi. Critical batch size revisited: A simple empirical approach to large-batch language model training. *Advances in Neural Information Processing Systems*, 38:116936–116959, 2026.
- 41 Mario Michael Krell, Matej Kosec, Sergio P Perez, and Andrew Fitzgibbon. Efficient sequence packing without cross-contamination: Accelerating large language models without impacting performance. *arXiv preprint arXiv:2107.02027*, 2022. URL <https://arxiv.org/abs/2107.02027v2>. Version 2, revised October 5, 2022.
- 42 Hao Liu, Matei Zaharia, and Pieter Abbeel. RingAttention with blockwise transformers for near-infinite context. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/1119587863e78451f080da2a768c4935-Abstract-Conference.html.
- 43 Linzheng Chai, Jian Yang, Shukai Liu, Wei Zhang, Liran Wang, Ke Jin, Tao Sun, Congnan Liu, Chenchen Zhang, Hualei Zhu, Jiaheng Liu, Xianjie Wu, Ge Zhang, Tianyu Liu, and Zhoujun Li. Multilingual multimodal software developer for code generation, 2025. URL <https://arxiv.org/abs/2507.08719>.
- 44 Fanglin Xu, Wei Zhang, Jian Yang, Guo Chen, Aishan Liu, Zhoujun Li, Xianglong Liu, and Bryan Dai. M2g-eval: Enhancing and evaluating multi-granularity multilingual code generation, 2025. URL <https://arxiv.org/abs/2512.22628>.
- 45 Jian Yang, Jiajun Zhang, Jiayi Yang, Ke Jin, Lei Zhang, Qiyao Peng, Ken Deng, Yibo Miao, Tianyu Liu, Zeyu Cui, Binyuan Hui, and Junyang Lin. Execrepobench: Multi-level executable code completion evaluation, 2024. URL <https://arxiv.org/abs/2412.11990>.
- 46 Jian Yang, Wei Zhang, Yizhi Li, Shawn Guo, Haowen Wang, Aishan Liu, Ge Zhang, Zili Wang, Zhoujun Li, Xianglong Liu, and Weifeng Lv. Codesimpleqa: Scaling factuality in code large language models, 2025. URL <https://arxiv.org/abs/2512.19424>.
- 47 Tianyu Zheng, Ge Zhang, Tianhao Shen, Xueling Liu, Bill Yuchen Lin, Jie Fu, Wenhui Chen, and Xiang Yue. Opencodeinterpreter: Integrating code generation with execution and refinement. *arXiv preprint arXiv:2402.14658*, 2024.
- 48 Xiang Yue, Tuney Zheng, Ge Zhang, and Wenhui Chen. Mammoth2: Scaling instructions from the web. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 90629–90660. Curran Associates, Inc., 2024. doi: 10.52202/079017-2877. URL <https://arxiv.org/abs/2402.14658>.

[//proceedings.neurips.cc/paper_files/paper/2024/file/a4ca07aa108036f80cbb5b82285fd4b1-Paper-Conference.pdf](https://proceedings.neurips.cc/paper_files/paper/2024/file/a4ca07aa108036f80cbb5b82285fd4b1-Paper-Conference.pdf).

- 49 Niklas Muennighoff, Qian Liu, Armel Zebaze, Qinkai Zheng, Binyuan Hui, Terry Yue Zhuo, Swayam Singh, Xiangru Tang, Leandro von Werra, and Shayne Longpre. Octopack: Instruction tuning code large language models, 2024. URL <https://arxiv.org/abs/2308.07124>.
- 50 Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, Tianyang Liu, Max Tian, Denis Kocetkov, Arthur Zucker, Younes Belkada, Zijian Wang, Qian Liu, Dmitry Abulkhanov, Indraneil Paul, Zhuang Li, Wen-Ding Li, Megan Risdal, Jia Li, Jian Zhu, Terry Yue Zhuo, Evgenii Zheltonozhskii, Nii Osa Osa Dade, Wenhao Yu, Lucas Krauß, Naman Jain, Yixuan Su, Xuanli He, Manan Dey, Edoardo Abati, Yekun Chai, Niklas Muennighoff, Xiangru Tang, Muhtasham Oblokulov, Christopher Akiki, Marc Marone, Chenghao Mou, Mayank Mishra, Alex Gu, Binyuan Hui, Tri Dao, Armel Zebaze, Olivier Dehaene, Nicolas Patry, Canwen Xu, Julian McAuley, Han Hu, Torsten Scholak, Sebastien Paquet, Jennifer Robinson, Carolyn Jane Anderson, Nicolas Chapados, Mostofa Patwary, Nima Tajbakhsh, Yacine Jernite, Carlos Muñoz Ferrandis, Lingming Zhang, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. Starcoder 2 and the stack v2: The next generation, 2024. URL <https://arxiv.org/abs/2402.19173>.
- 51 Siwei Wu, Yizhi Li, Yuyang Song, Wei Zhang, Yang Wang, Riza Batista-Navarro, Xian Yang, Mingjie Tang, Bryan Dai, Jian Yang, and Chenghua Lin. Large-scale terminal agentic trajectory generation from dockerized environments, 2026. URL <https://arxiv.org/abs/2602.01244>.
- 52 U.S. Bureau of Labor Statistics. 2018 standard occupational classification system, 2018. URL https://www.bls.gov/soc/2018/major_groups.htm.
- 53 Wenhan Ma, Jianyu Wei, Liang Zhao, Hailin Zhang, Bangjun Xiao, Lei Li, Qibin Yang, Bofei Gao, Yudong Wang, Rang Li, Jinhao Dong, Zhifang Sui, and Fuli Luo. Mopd: Multi-teacher on-policy distillation for capability integration in llm post-training, 2026. URL <https://arxiv.org/abs/2606.30406>.
- 54 Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. MiniLLM: Knowledge distillation of large language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2306.08543v3>.
- 55 Wenqi Huang, Charley Lee, Leonard Tng, and Serena Ge. Deepswe: Measuring frontier coding agents on original, long-horizon engineering tasks. *arXiv preprint arXiv:2607.07946*, 2026. URL <https://arxiv.org/abs/2607.07946>.
- 56 Wenqi Huang and Peter Jiang. Deepswe v1.1: A cleaner, more reproducible benchmark for frontier coding agents, 2026. URL <https://deepswe.datacurve.ai/blog/deepswe-v1-1>.
- 57 Kabir Khandpur, Kilian Lieret, Carlos E. Jimenez, Ofir Press, and John Yang. Swe-bench multilingual, 2025. URL <https://www.swebench.com/multilingual.html>.
- 58 John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. Swe-smith: Scaling data for software engineering agents, 2025. URL <https://arxiv.org/abs/2504.21798>.

- 59 Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, et al. Swe-bench pro: Can ai agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941*, 2025. URL <https://arxiv.org/abs/2509.16941>.
- 60 Mohit Raghavendra, Soham Dan, Miguel Romero Calvo, Yannis Yiming He, Johannes Baptist Mols, Gautam Anand, Cole McCollum, Edgar Arakelyan, Vijay Bharadwaj, Andrew Park, Jeff Da, MohammadHossein Rezaei, Bing Liu, Brad Kenstler, and Yunzhong He. SWE Atlas: Benchmarking coding agents beyond issue resolution, 2026. URL <https://arxiv.org/abs/2605.08366>.
- 61 Jingzhe Ding, Shengda Long, Changxin Pu, Huan Zhou, Hongwan Gao, et al. NL2repo-bench: Towards long-horizon repository generation evaluation of coding agents. In *International Conference on Machine Learning*, 2026. URL <https://icml.cc/virtual/2026/poster/60772>.
- 62 John Yang, Kilian Lieret, Jeffrey Ma, Parth Thakkar, Dmitrii Pedchenko, Sten Sootla, Emily McMilin, Pengcheng Yin, Rui Hou, Gabriel Synnaeve, Diyi Yang, and Ofir Press. Programbench: Can language models rebuild programs from scratch?, 2026. URL <https://arxiv.org/abs/2605.03546>.
- 63 Zhun Wang, Tianneng Shi, Jingxuan He, Matthew Cai, Jialin Zhang, and Dawn Song. Cybergym: Evaluating AI agents’ real-world cybersecurity capabilities at scale. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://arxiv.org/abs/2506.02548>.
- 64 Xiangyi Li, Yimin Liu, Wenbo Chen, Bingran You, Zonglin Di, Yifeng He, et al. Skills-bench: Benchmarking how well agent skills work across diverse tasks. *arXiv preprint arXiv:2602.12670*, 2026. URL <https://arxiv.org/abs/2602.12670>.
- 65 Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=a7Qa4CcHak>.
- 66 Harbor Framework Team. Terminal-bench 2.1, 2026. URL <https://github.com/harbor-framework/terminal-bench-2-1>.
- 67 Terminal-Bench Team. Terminal-bench 3.0, 2026. URL <https://www.tbench.ai/news/terminal-bench-3-0>.
- 68 Terminal-Bench Team. Terminal-bench 4.0, 2026. URL <https://www.tbench.ai/news/terminal-bench-4-0>.
- 69 Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, Johannes Heidecke, Amelia Glaese, and Tejal Patwardhan. PaperBench: Evaluating AI’s ability to replicate AI research. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267, pages 56843–56873. PMLR, 2025. URL <https://proceedings.mlr.press/v267/starace25a.html>.
- 70 David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*, 2024. URL <https://arxiv.org/abs/2311.12022>.

- 71 Long Phan, Alice Gatti, Ziwen Han, Nathaniel Li, Josephina Hu, Hugh Zhang, Chen Bo Calvin Zhang, Mohamed Shaaban, John Ling, Sean Shi, et al. Humanity’s last exam, 2025. URL <https://arxiv.org/abs/2501.14249>.
- 72 Tejal Patwardhan, Rachel Dias, Elizabeth Proehl, Grace Kim, Michele Wang, Olivia Watkins, Simón Posada Fishman, Marwan Aljubeh, Phoebe Thacker, Laurance Fauconnet, Natalie S. Kim, Patrick Chao, Samuel Miserendino, Gildas Chabot, David Li, Michael Sharman, Alexandra Barr, Amelia Glaese, and Jerry Tworek. GDPval: Evaluating AI model performance on real-world economically valuable tasks, 2025. URL <https://arxiv.org/abs/2510.04374>. ICLR 2026.
- 73 Junlong Li, Wenshuo Zhao, Jian Zhao, Weihao Zeng, Haoze Wu, Xiaochen Wang, Rui Ge, Yuxuan Cao, Yuzhen Huang, Wei Liu, Junteng Liu, Zhaochen Su, Yiyang Guo, Fan Zhou, Lueyang Zhang, Juan Michelini, Xingyao Wang, Xiang Yue, Shuyan Zhou, Graham Neubig, and Junxian He. The tool decathlon: Benchmarking language agents for diverse, realistic, and long-horizon task execution, 2025. URL <https://arxiv.org/abs/2510.25726>. ICLR 2026.
- 74 Bowen Ye, Rang Li, Qibin Yang, Yuanxin Liu, Linli Yao, Hanglong Lv, Zhihui Xie, Chenxin An, Lei Li, Lingpeng Kong, Qi Liu, Zhifang Sui, and Tong Yang. Claw-eval: Towards trustworthy evaluation of autonomous agents, 2026. URL <https://arxiv.org/abs/2604.06132>.
- 75 Daniel Shepard and Robin Salimans. Automationbench, 2026. URL <https://arxiv.org/abs/2604.18934>.
- 76 Yuetai Li, Yichen Feng, Zhangchen Xu, Zixian Ma, Kaiyuan Zheng, Fengqing Jiang, Xinghua Sun, Rulin Shao, Zichen Chen, Yue Huang, Xinyang Han, Brian Lee, Kayla Xu, Shenglai Zeng, Hang Hua, Xiangliang Zhang, Basel Alomair, Ranjay Krishna, Luke Zettlemoyer, Pang Wei Koh, Bhaskar Ramasubramanian, Luyao Niu, Xiang Yue, and Radha Poovendran. Jobbench: Aligning agent work with human will, 2026. URL <https://arxiv.org/abs/2605.26329>.
- 77 Yiyao Sun, Xinyang Han, Weichen Zhang, Yuanbo Pang, Tianyu Wang, Yuhao Cao, Yixiao Huang, Chris Duroiu, Haoyun Zhang, Jeffrey Lin, Weishu Zhang, Tyler Zeng, Ying Yan, Bo Liu, Hanson Wen, Mingyang Xu, Xiaoyuan Liu, Zimeng Chen, Weiyan Shi, Amanda Dsouza, et al. Agents’ last exam, 2026. URL <https://arxiv.org/abs/2606.05405>.
- 78 Quan Shi, Alexandra Zyttek, Pedram Razavi, Karthik Narasimhan, and Victor Barres. τ -Knowledge: Evaluating conversational agents over unstructured knowledge, 2026. URL <https://arxiv.org/abs/2603.04370>.
- 79 Sierra. τ^3 -Bench: Advancing agent benchmarking to knowledge and voice, 2026. URL <https://sierra.ai/blog/bench-advancing-agent-benchmarking-to-knowledge-and-voice>.
- 80 Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, Isa Fulford, Hyung Won Chung, Alex Tachard Passos, William Fedus, and Amelia Glaese. BrowseComp: A simple yet challenging benchmark for browsing agents, 2025. URL <https://arxiv.org/abs/2504.12516>.
- 81 Ryan Z. Y. Wong, Jiawei Wang, Junjie Zhao, Li Chen, Yan Gao, Long Zhang, Xuan Zhou, Zuo Wang, Kai Xiang, Ge Zhang, Wenhao Huang, Yang Wang, and Ke Wang. Widesearch: Benchmarking agentic broad info-seeking, 2025. URL <https://arxiv.org/abs/2508.07999>. ICLR 2026.

- 82 Rebecca Soskin Hicks, Mikhail Trofimov, Dominick Lim, Rahul K. Arora, Foivos Tsimpourlas, Preston Bowman, Michael Sharman, Chi Tong, Kavin Karthik, Arnav Dugar, Akshay Jagadeesh, Khaled Saab, Johannes Heidecke, Ashley Alexander, Nate Gross, and Karan Singhal. Healthbench professional: Evaluating large language models on real clinician chats, 2026. URL <https://arxiv.org/abs/2604.27470>.
- 83 Thang Luong, Dawsen Hwang, Hoang H. Nguyen, Golnaz Ghiasi, Yuri Chervonyi, Insuk Seo, Junsu Kim, Garrett Bingham, Jonathan Lee, Swaroop Mishra, Alex Zhai, Clara Huiyi Hu, Henryk Michalewski, Jimin Kim, Jeonghyun Ahn, Junhwi Bae, Xingyou Song, Trieu H. Trinh, Quoc V. Le, and Junehyuk Jung. Towards robust mathematical reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025. URL <https://aclanthology.org/2025.emnlp-main.1794/>.
- 84 Minhui Zhu, Mingyang Tian, Xiaocheng Yang, Tianci Zhou, Lifan Yuan, Penghao Zhu, Eli Chertkov, Shengyan Liu, Yufeng Du, Ziming Ji, et al. Probing the critical point (critpt) of ai reasoning: a frontier physics research benchmark. *arXiv preprint arXiv:2509.26574*, 2025.
- 85 SWE-agent Team. mini-swe-agent, 2025. URL <https://github.com/SWE-agent/mini-swe-agent>.
- 86 Marina Favaro and Jack Clark. When AI builds itself. Anthropic Institute, 2026. URL <https://www.anthropic.com/institute/recursive-self-improvement>. Accessed September 20, 2026.
- 87 Mingguang Chen, Licheng Wang, and Bo Qu. Recursive self-improvement in AI: From bounded self-refinement to autonomous research loops, 2026. URL <https://arxiv.org/abs/2607.07663v2>. Version 2, September 6, 2026.
- 88 Jiaxin Wen, Liang Qiu, Joe Benton, Jan Hendrik Kirchner, and Jan Leike. Automated weak-to-strong researcher. Anthropic Alignment Science Blog, April 2026. URL <https://alignment.anthropic.com/2026/automated-w2s-researcher/>. Accompanying Anthropic Research announcement dated April 14, 2026; accessed September 20, 2026.
- 89 Zihan Tan, Leixin Sun, Zitong Shi, Yitao Liu, Jiajun Wu, Nathaniel Brooks, Jiaru Qian, Xiaoran Shang, Suyuan Huang, Yi Ding, Yangxu Liao, Mukai Li, Qiushi Sun, Shudong Liu, Xuankun Rong, Xiaohang Yu, Zhuo Chen, Hejia Geng, Chenxin Li, Aozhou Wang, Zengji Tu, Robert Tang, Yuxin Zhan, Eric Jiang, Yuxin Wu, Jianqing Zhang, Xiao Liang, Fang Wu, Haochi Zhang, Alexander Marlow, and Guancheng Wan. MetaRSI / RSI2: A Meta-Recursive Self-Improving System for Recursive Self-Improving Systems Themselves, 2026. URL <https://arxiv.org/abs/2609.06396v2>. Version 2, September 9, 2026; accessed September 20, 2026.
- 90 OpenAI. Harness engineering: leveraging Codex in an agent-first world. OpenAI blog, 2026. URL <https://openai.com/index/harness-engineering/>. Accessed September 20, 2026.
- 91 Shuhan Xue, Jianyuan Zhong, Ziyuan Nan, Wenbin Li, Zhaochen Yu, Jinchao Ding, Qiang Gao, Pengyu Zhan, Yuntong Zhang, Tian Cheng, Zhenfei Yin, Yingcheng Wu, and Ling Yang. ScienceBuddy: Recursive-in-Recursive Self-Improvement for Interactive Scientific Agents, September 2026. URL <https://arxiv.org/abs/2609.17523v1>. Version 1, September 15, 2026; accessed September 20, 2026.
- 92 Adam Zweiger, Jyothish Pari, Han Guo, Yoon Kim, and Pulkit Agrawal. Self-adapting language models. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2025*, 2025.

- 93 Siwei Wu, Jincheng Ren, Yizhi Li, Haau-Sing Li, Chengran Yang, Yuxuan Zhang, Weicheng Gu, Jian Yang, Riza Batista-Navarro, Chuanyi Zhang, Xianglong Liu, Ming Zhou, Bryan Dai, and Chenghua Lin. ModularRSI: Modular and Generalizable Recursive Harness Self-Improvement, September 2026. URL <https://arxiv.org/abs/2609.14857v1>. Version 1, September 14, 2026; accessed September 20, 2026.
- 94 OpenAI. Research acceleration: The view inside OpenAI. OpenAI blog, September 2026. URL <https://openai.com/index/research-acceleration-view-inside-openai/>. Accessed September 20, 2026.
- 95 Haozhe Liu, Tian Ye, Sensen Gao, Qihang Cao, Yitong Li, Mingchen Zhuge, Duomin Wang, Ruihua Zhang, Ping Luo, Jiawang Bian, Lei Zhu, Ligeng Zhu, Enze Xie, and Song Han. SoL-Pi: Recursively Scaling Auto-Research Loops for Efficient Agent Harness, September 2026. URL <https://arxiv.org/abs/2609.20519v1>. Version 1, September 17, 2026; accessed September 20, 2026.

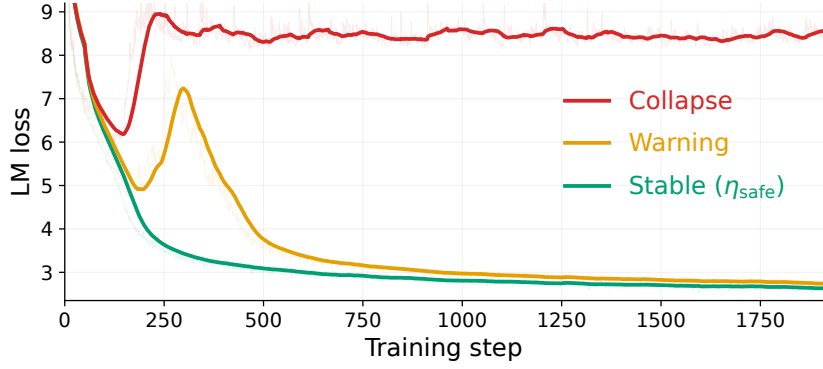


Figure 9. Representative training-loss trajectories for the model under stable, warning, and collapsed training regimes.

Module	Initialization Std.	Learning-Rate
Embedding	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Linear	$\mathcal{O}(d^{-1/2})$	$\mathcal{O}(d^{-1/2}L^{-1/4})$
RMSNorm	–	$\mathcal{O}(L^{-1/4})$
LM head	$\mathcal{O}(d^{-1})$	$\mathcal{O}(d^{-1})$

Table 3. Width- and depth-dependent μ P scaling rules used for different modules. Here, d denotes the hidden width and L denotes the model depth.

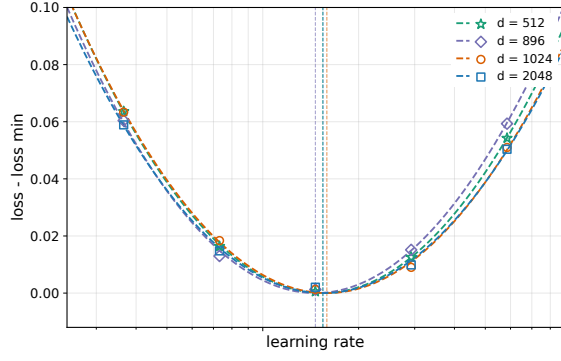
A. Pre-Training

Safe learning rate. We perform a grid search over learning rates and classify the resulting training-loss trajectories as stable, warning, or collapsed, as illustrated in Figure 9. Stable runs show consistently decreasing loss; warning runs exhibit a transient rebound before recovering, while collapsed runs rebound and remain at high loss. We define the safe learning rate, η_{safe} , as the largest tested learning rate classified as stable.

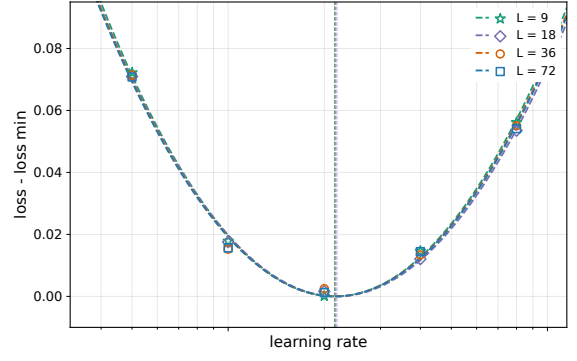
μ P scaling factors. Table 3 summarizes the width- and depth-dependent scaling rules used to train IQuest-Q1. Following CompleteP [33], we apply an output scaling factor of $L^{-1/2}$ to both the attention and FFN branches. For the attention branch, this factor is incorporated into the output RMSNorm. For the FFN branch, it is applied directly to the branch output.

As shown in Figure 10, these scaling rules enable consistent transfer of the optimal learning rate across models with different widths and depths.

SWA linear context parallelism with partial halo attention. Figure 11 illustrates the setting $0 < H < S$, where S denotes the number of tokens in each contiguous sequence shard and H denotes the number of preceding tokens accessible to each query. In panel (b), query and key positions are measured relative to the start of the local shard. For a query at position i , valid key positions satisfy $i - H \leq j \leq i$; negative key positions belong to the preceding shard. The horizontal dashed line separates boundary queries from queries whose attention windows lie entirely within the local shard. The pairs (O_ℓ, L_ℓ) and (O_h, L_h) denote the normalized attention outputs and corresponding per-query, per-head log-sum-exp statistics for the local and halo contributions, respectively. Panel (c) depicts execution dependencies schematically.



(a) Across different width.

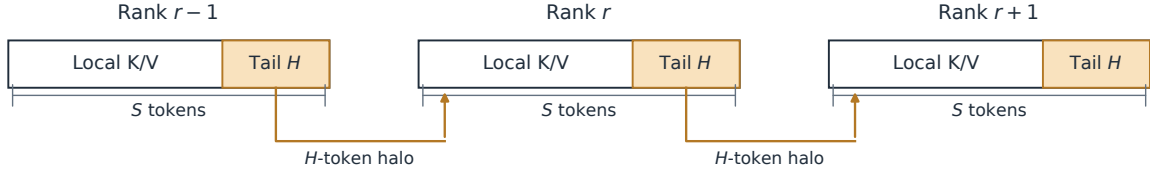


(b) Across different depth.

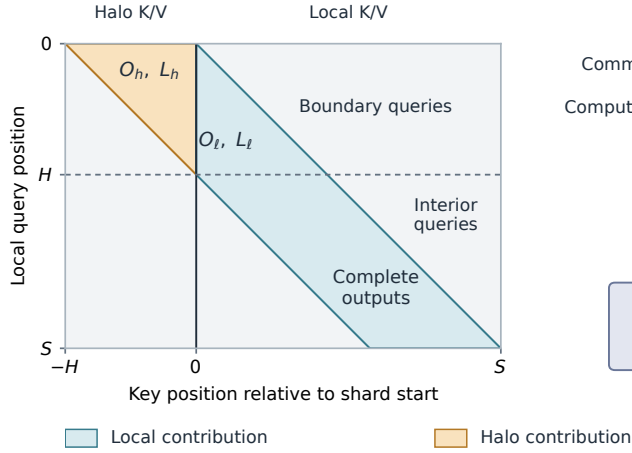
Figure 10. Optimal learning rate transfer across model widths and depths under μP scaling.

Recursive Multi-token Prediction. Figure 12 illustrates the workflow of recursive multi-token prediction. We adopt the MTP architecture of DeepSeek-V3 [35], which concatenates a hidden representation with a token embedding to form the input to the MTP block. At the first drafting step, the hidden representation comes from the final layer of the backbone model. At each subsequent step, we concatenate the previous step’s output hidden representation with the embedding of the token predicted at that step and feed the result back into the same MTP block, enabling recursive draft generation with shared parameters.

(a) Contiguous partitioning and halo exchange



(b) Local and halo contributions



(c) Forward schedule and merge

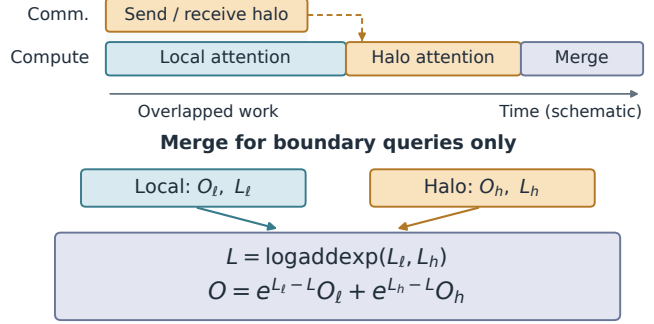


Figure 11. Overlapping halo exchange with partial attention computation.

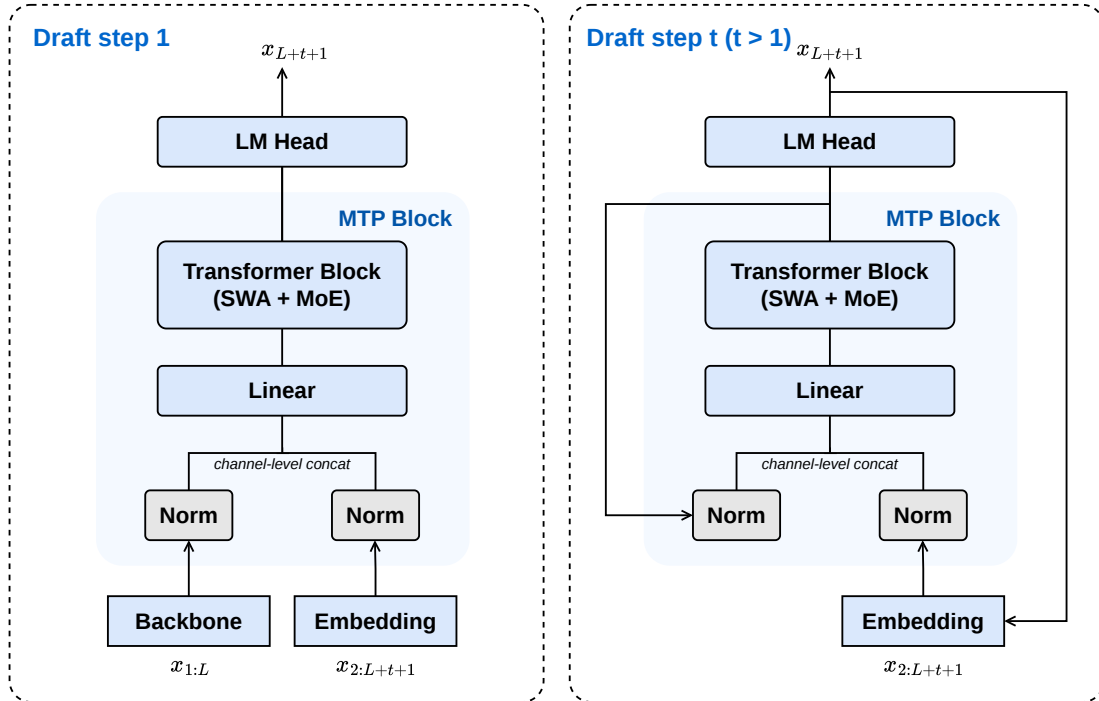


Figure 12. Illustration of recursive multi-token prediction. Here, L denotes the input sequence length and t denotes the drafting step.